CAB432 Assignment 3 2026

DocuMind AI: a serverless document-intelligence service

Name: Tri Cuong Dao
Student ID: n12136514
Email: n12136514@qut.edu.au
Deployment region: Asia Pacific (Sydney), ap-southeast-2, shared CAB432 account 901444280953
Repository: Assessment 3/ (backend, frontend), submitted alongside this report

Declaration of AI use

CAB432 authorises generative AI for this assessment on the condition that the author stays accountable for the result. I used AI tools as an assistant in three areas: drafting parts of the application code, working through defects in the ingestion and caching paths, and editing the wording of this report. I chose the architecture and made the design decisions, and I ran the deployed system myself to produce every measurement in Sections 3, 4 and 5.

I checked each claim against the running system before submitting. Where a tool gave me text I did not agree with, I changed it or removed it. Several tools asserted things about this account that turned out to be false, including availability of managed scaling and the behaviour of the cache, and those corrections are recorded in the sections where they mattered.

Executive Summary

DocuMind AI answers questions about PDFs. A user drops documents onto a web app and, minutes later, asks about them in plain language. Every answer cites the passage it came from. The workload is bursty and knowledge-heavy: a study group uploads a semester of readings at midnight, or a team batch-loads reports at the end of a sprint. Demand spikes hard, sits idle for long stretches, and the documents matter for a short window. Uploads, extracted text, summaries and job records expire after 24 hours, and their vectors are swept from the index.

Two kinds of work run through this system. Ingestion is heavy, asynchronous and bursty. Question answering is latency-sensitive and repetitive, because people ask the same few questions about the same few documents. The design separates the two: a durable queue and a pool of container workers absorb ingestion bursts, while a stateless API serves queries through a two-tier cache, a managed vector index and a foundation model.

DocuMind covers the cloud patterns the brief asks for. Unstructured storage holds PDFs, extracted text and summaries. A single-table metadata store uses TTL for garbage collection. Caching runs in two tiers, an in-process LRU and a shared tier ahead of every model call. A message queue carries work, with a dead-letter queue for fault isolation. An API gateway exposes the microservice API. Workers pull from the queue by long-polling. The query API runs serverless, ingestion runs in containers, and the container pool autoscales on queue backlog by metric, which leaves Lambda's built-in scaling uncounted. Storage notifications drive the flow. Roles follow least privilege. Logs, metrics and a dashboard carry observability. Managed AI services supply embeddings and answer synthesis. Storage TTLs and lifecycle rules handle expiry. Handlers stay stateless, and bounded retries, visibility timeouts and idempotent writes cover resilience.

Evidence from the deployed system backs each claim. A 30-document burst took the worker pool from 1 to 6 tasks inside one 60-second control tick, then back to zero when the work ran out (Section 3, Figure 3.2). The caching experiment measured a 2 408 ms median origin answer against 0 ms for an in-process cache hit, with a 46 ms median round trip across 112 counted requests (Figure 5.1, Section B.2). Poisoned documents landed in the dead-letter queue without taking a worker down (Section 4). The end-to-end functional checklist passed 14 of 14. Monthly cost at 50 concurrent users sits at ≈US$16.68 on-demand, or ≈US$8.98 with Fargate Spot, and the always-on controller task plus the observability stack make up most of it (Section 5).

Section 1 sets out the problem, the requirements and the user-facing behaviour. Section 2 gives the vendor-agnostic design pattern by pattern, with its trade-offs and its limits. Section 3 maps that design onto AWS and records the account constraints that shaped it. Section 4 follows data through the system, including schemas, sequence flows and failure handling. Section 5 covers cost and sustainability. Section 6 reflects on independent learning. Sections 7 and 8 close with conclusions and references, and the appendices carry the supporting evidence.

Table of Contents

1. Introduction. 1.1 Problem statement and motivation. 1.2 Users and user stories. 1.3 Functional requirements. 1.4 Non-functional requirements. 1.5 Scope and constraints. 1.6 Front-end walkthrough.
2. Vendor-agnostic architecture. 2.1 Design overview. 2.2 Pattern selection and justification. 2.3 Asynchronous ingestion and worker-pull distribution. 2.4 Scaling reflection. 2.5 Caching reflection. 2.6 Identity, security and configuration. 2.7 Observability. 2.8 Reliability, failure handling and backpressure. 2.9 Trade-offs and alternatives considered. 2.10 Limitations and how they would be addressed.
3. Vendor-specific AWS implementation. 3.1 Service mapping and the AWS architecture. 3.2 Why these AWS services. 3.3 Identity and least privilege in practice. 3.4 Networking. 3.5 Wiring the event path. 3.6 Autoscaling implementation. 3.7 Observability implementation. 3.8 Account constraints and adaptations.
4. Application implementation. 4.1 Data model and schemas. 4.2 Ingestion workflow. 4.3 Query workflow. 4.4 Interfaces: REST API and MCP tools. 4.5 Failure modes and recovery. 4.6 Behaviour under load.
5. Cost and sustainability. 5.1 Usage model for 50 concurrent users. 5.2 Monthly cost breakdown. 5.3 How scaling affects cost. 5.4 Sustainability analysis.
6. Reflection on independent learning (ULO4). 7. Conclusion. 8. References. Appendices. A: cloud infrastructure declaration. B: experiment methodology and measurements. C: API and MCP reference. D: pipeline parameters and prompt template.

---

1. Introduction

1.1 Problem statement and motivation

Reading a pile of PDFs takes time. Searching them goes worse: keyword search misses paraphrase, and the tools that claim to answer questions about a document usually cost money, harvest your data, or both. DocuMind AI takes a narrow slice of that problem. You upload PDFs, ask questions, get answers with citations, and everything expires on its own. The 24-hour retention window is a product decision. It removes document housekeeping, it caps storage cost, and it suits material that should not sit on a server indefinitely.

Three properties of this workload make it a cloud problem rather than a local script.

1. **Bursty, fragmented ingestion.** Users push batches of 10 to 40 files in a few minutes, then nothing for hours. Sizing for the peak wastes money. Sizing for the average makes people wait. That combination calls for a queue and a worker pool that grows and shrinks, which is why ingestion runs asynchronously.
2. **Latency-sensitive, repetitive interaction.** Once documents are ready, people ask a small set of questions over and over ("what are the constraints?", "summarise section 3"). An origin answer costs one embedding call, one vector search and one generation call. That means seconds of waiting, and generation is the most expensive operation in the system. A cache turns most interaction into a response under 15 ms and removes the repeated model spend.
3. **Managed AI, not hosted models.** The system needs embeddings and text generation. It does not need anyone to run GPUs. Managed foundation-model APIs fit this scale: pay per token, no idle accelerator.

The system therefore has to cost nothing when idle, absorb bursts without dropping work, and keep interactive latency low. Those three constraints drive every decision in Section 2.

1.2 Users and user stories

The main persona is a student or knowledge worker who receives PDFs and needs answers fast. The second is a developer wiring document Q&A into another tool.

ID	As a…	I want to…	So that…
US-1	user	upload several PDFs at once by dragging them onto a page	I don't queue them one by one
US-2	user	see, for each file, where it is in processing (queued → processing → ready)	I know when I can ask questions
US-3	user	ask a question in plain language and get an answer with citations	I can verify the answer against the source
US-4	user	ask the same question again and get an instant response	repeated lookup feels native, not slow
US-5	user	trust that my documents disappear after a day	sensitive material doesn't accumulate
US-6	developer	call the same Q&A capability from tools/agents via a standard interface	I can embed DocuMind into other workflows
US-7	operator	see queue depth, worker count and error state at a glance	I can tell healthy from unhealthy in seconds

1.3 Functional requirements

ID	Requirement	Verification
FR-1	The API issues pre-signed object-storage upload URLs (15-minute validity); file bytes never transit the API	Smoke test: presign → direct PUT → 200
FR-2	An upload automatically triggers asynchronous processing: text extraction, semantic chunking, embedding and summarisation	End-to-end job observed CREATED → EMBEDDING → COMPLETED in ≈29 s
FR-3	Job status is queryable per document (CREATED, PROCESSING, EMBEDDING, COMPLETED, FAILED)	Status transitions observed on every processed document
FR-4	POST /api/query answers questions over all currently processed documents with per-source citations (file, chunk, similarity)	Live query returned 4 cited sources
FR-5	Answers are cached in two tiers; responses carry X-Cache: HIT-L1 / HIT-L2 / MISS and a latency figure	Every repeat (50/50) and burst duplicate (12/12) served from cache in the experiment (Section B.2)
FR-6	Every stored artifact expires automatically ≈24 h after creation (job rows, uploaded PDFs, extracted text, summaries)	TTL attributes present; lifecycle rule configured; a sweep deletes vectors when their documents expire, because the vector index has no native TTL
FR-7	Poison documents (corrupt, password-protected) are isolated without crashing workers and are visible as FAILED	DLQ experiment: message routed after 3 receives, worker unaffected

ID	Requirement	Verification
FR-8	An MCP tool endpoint exposes document search and question answering to LLM agents	tools/list returns 3 tools; search_documents exercised live
FR-9	Admin operations (cache purge, forcing reprocessing) are protected by a secret key	Admin call rejected without X-API-Key; accepted with it
FR-10	Static front-end is served from the same origin as the API (no CORS complexity for users)	UI loads from the service URL

1.4 Non-functional requirements

ID	Requirement	Target	Measured
NFR-1	Query latency, cached	p95 < 50 ms	46 ms median round-trip including network (0 ms server-side at tier 1)
NFR-2	Query latency, origin (embed + retrieve + generate)	p95 < 8 s	3 339 ms server-side p95 (p50 2 408 ms)
NFR-3	Elasticity	absorb a 30-document burst; scale worker pool 1 → ≥4 within 2 minutes	1 → 6 within one 60 s tick
NFR-4	Idle cost	≈ 0 compute when unused; no always-on workers	Worker service desired 0 between bursts
NFR-5	Data lifetime	Every stored artifact expires ≤ 24 h (vectors swept separately)	TTL + lifecycle rule; vector sweep script
NFR-6	Durability	No lost uploads: every accepted upload eventually processes or lands in a dead-letter queue	30/30 burst documents processed; 1/1 poison isolated
NFR-7	Security	Least-privilege workload roles; no credentials in code; public access blocked at the storage buckets	Verified via IAM policy review and bucket policies
NFR-8	Cost at 50 concurrent users	Same order of magnitude as US$10/month (≤ US$25)	≈US$16.68/month on-demand; ≈US$8.98 at 70 % Spot

1.5 Scope and constraints

The scope is deliberately narrow. One region, ap-southeast-2. English PDFs up to 25 MB. Answers come only from uploaded documents, with no general-knowledge fallback. The demo front-end serves a single workspace, and authentication beyond the admin key stays out of scope. The deployment runs in the **CAB432 shared teaching account**, whose guardrails shaped the implementation and are set out in Section 3.8: a shared API gateway with additive path prefixes, a per-account limit of one gateway, an allow-list of foundation models, and a tag-conditional IAM policy that permits resource creation only when resources carry purpose=assessment 3 and the student's qut-username tag. Three managed capabilities went unused. An edge CDN was possible, since CloudFront is available in this account, but the deployment serves the SPA from the API origin with immutable asset headers (Section 2.10). A dedicated in-memory cache service and managed target-tracking scaling also went unused; Section 3.6 implements the control law directly. Section 2 presents the design as it would run in an unrestricted account. Section 3 records where the implementation adapts.

1.6 Front-end walkthrough

The front-end does a job rather than decorating one. It exercises the architecture: upload through pre-signed URLs, watch the pipeline, feel the cache.

Figure 1.1. Upload zone. Drag and drop takes multiple PDFs, then uploads each straight to object storage through the pre-signed URL the API issued. The browser shows per-file progress, and the API never touches file bytes.

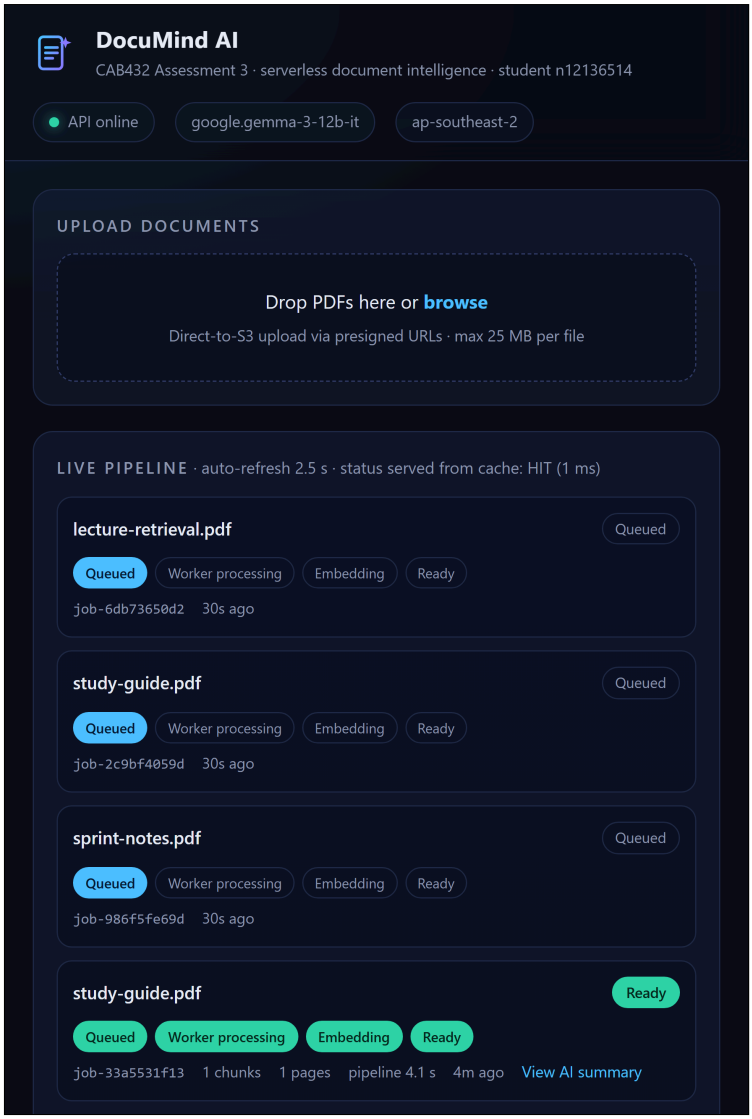


Figure 1.1. Upload zone with a batch of documents being submitted

Figure 1.2. Live pipeline monitor. Each document reports its state as it moves through queue, worker, embedding and ready. The status endpoint drives it, and a short-TTL cache keeps that polling cheap.

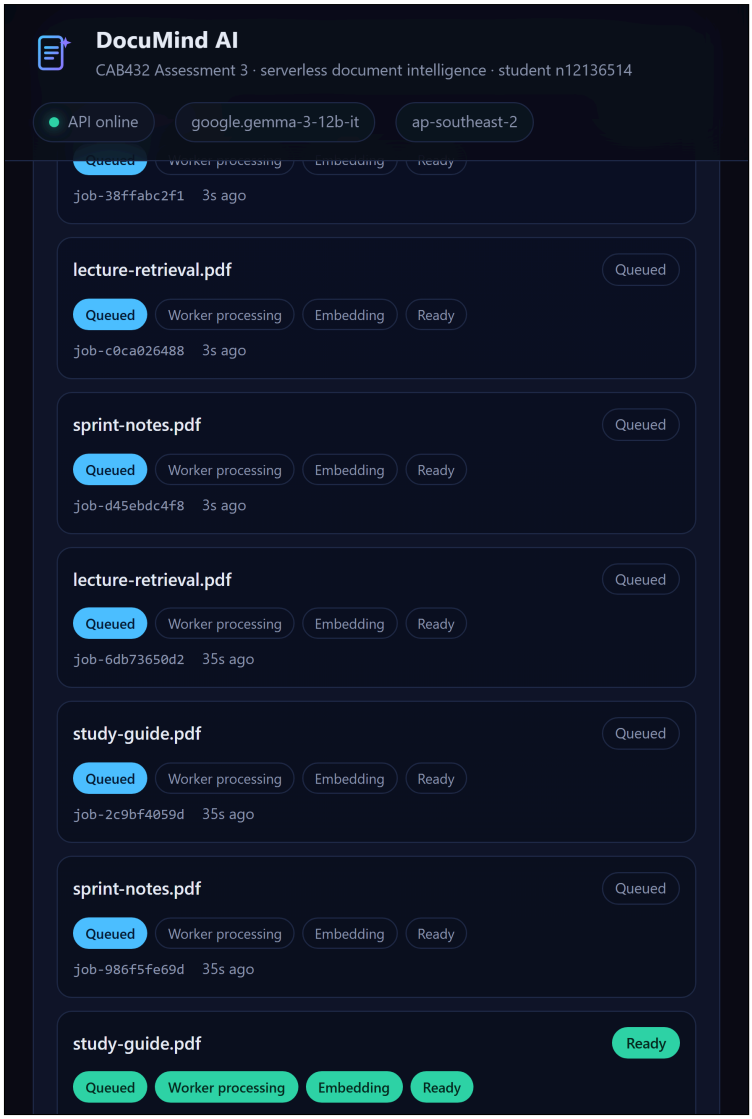


Figure 1.2. Live pipeline monitor showing documents at different stages

Figure 1.3. Cited Q&A chat. Answers cite sources as pills. Each answer carries a latency and cache badge (Cache hit (L1 memory) · 0 ms against Origin generation · 2.5 s), which shows the caching behaviour without opening developer tools.

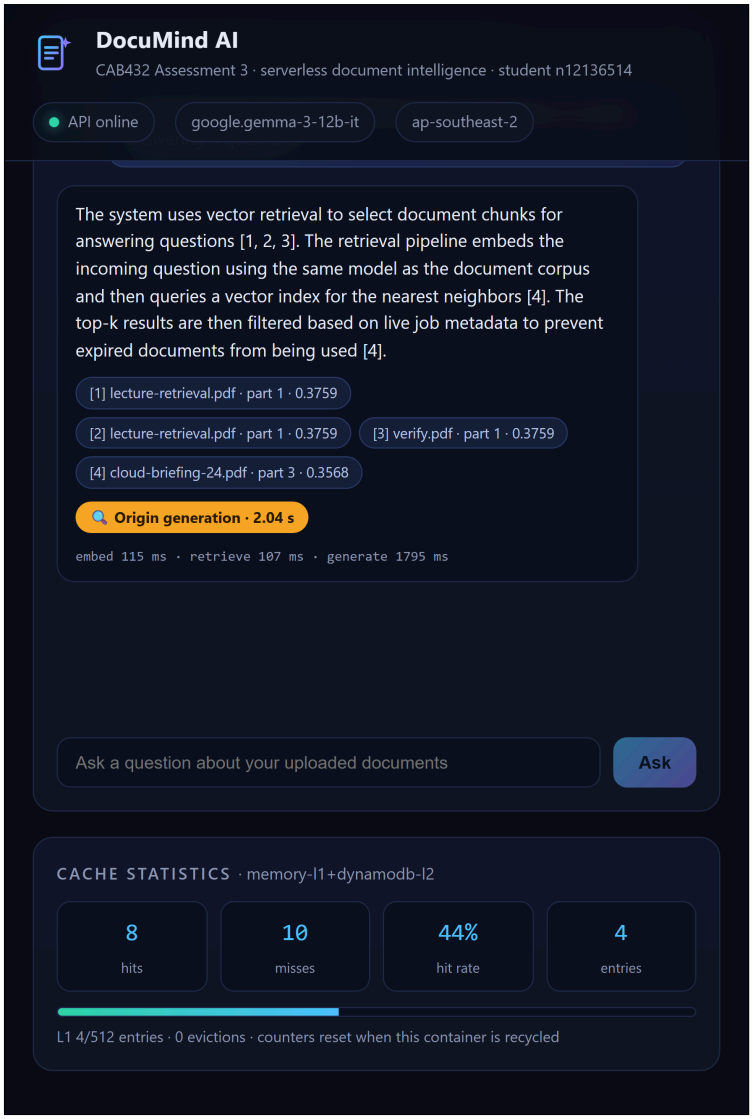


Figure 1.3. Chat pane with citations and the cache/latency badge

Figure 1.4. Cache statistics panel. Live L1 hit and miss counters, hit rate, entry count and the backend description, read from GET /api/cache/stats. Queue depth and worker count belong to the CloudWatch dashboard (Section 3.7). The SPA leaves them out, and the same short-TTL cache keeps its polling cheap.

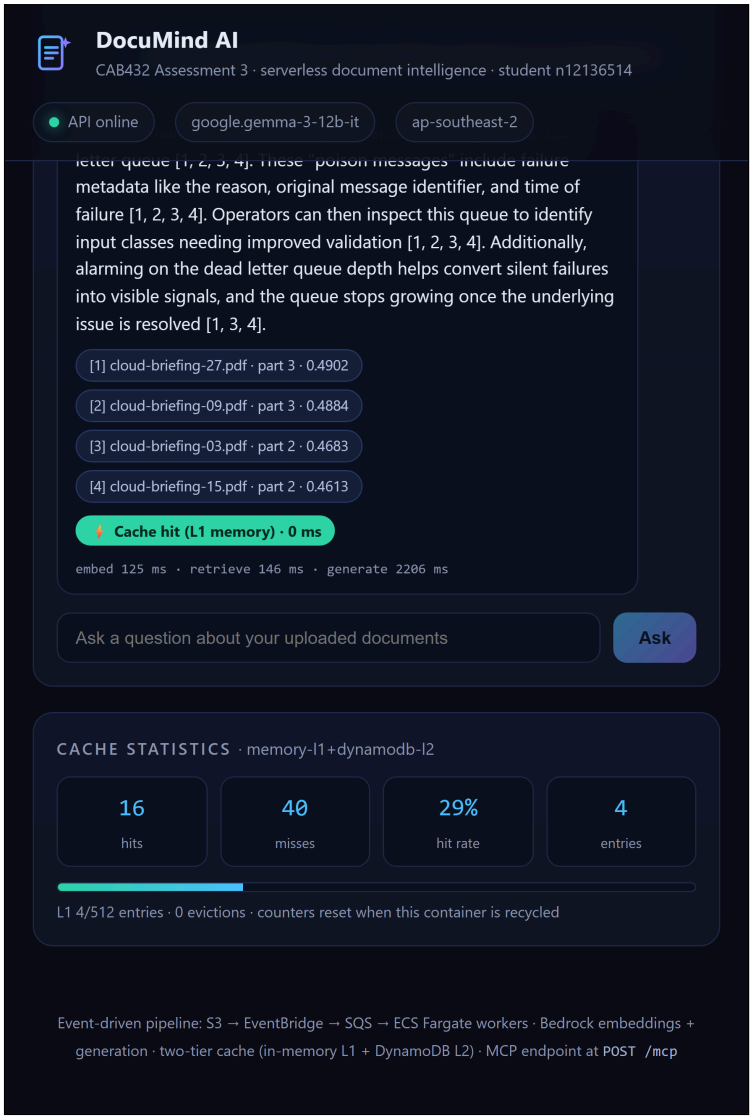


Figure 1.4. Operations panel with pipeline and cache statistics

2. Vendor-agnostic architecture

2.1 Design overview

Figure 2.1. Vendor-agnostic architecture. The design avoids vendor products. Every box maps to a category of managed service, and Section 3.1 gives the AWS mapping.

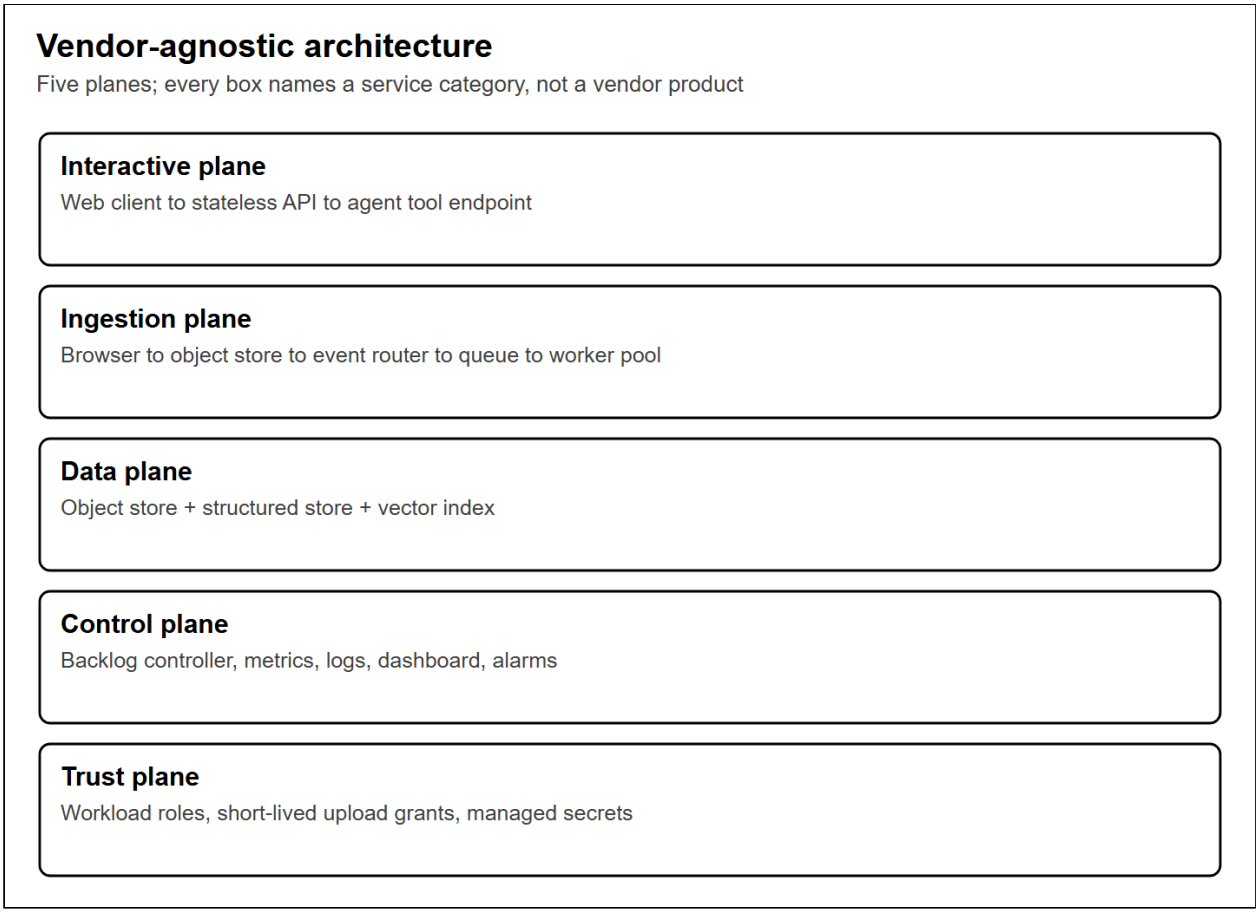


Figure 2.1. Vendor-agnostic architecture of DocuMind AI

Five planes do the work here.

Interactive plane. A web client talks to a stateless API tier over HTTPS and JSON. The API does three things: issues short-lived pre-signed upload URLs, answers queries, and exposes a tool interface for agents. It holds no durable state, because everything lives in the data plane. Any instance can die at any moment and nothing is lost.

Ingestion plane. The browser uploads file bytes straight to object storage through a pre-signed URL, so the API never touches payloads. The object store emits an object-created event. An event router filters for the uploads prefix and forwards it to a durable queue. Workers pull messages at their own pace, do the work against the model service (text extraction, chunking, embedding, summarisation), and write artifacts back to the stores. A dead-letter queue takes whatever a worker cannot process.

Data plane. Three storage shapes, one per shape of data. An object store holds unstructured blobs: raw PDFs, extracted text, summaries. A structured store holds small mutable records: job states and cached answers. A vector index holds embeddings alongside their source text. All three bill per request or per byte, and all three expire data on their own through TTL records and lifecycle rules.

Control plane. A controller reads queue backlog every 60 seconds and sets the worker pool's desired size from a target-value control law. Monitoring covers every component with metrics, logs, dashboards and alarms. It raises the two alarms that matter: dead-letter depth and sustained backlog.

Trust plane. Identity rests on workloads. Each component runs under a role scoped to the resources it needs. Clients hold short-lived upload grants for a single object instead of credentials, and secrets live in a managed store. No component authenticates with a static key.

2.2 Pattern selection and justification

The assignment requires at least eight pattern categories, with auto-scaling and caching explicitly demonstrated (and Lambda's built-in scaling explicitly excluded). DocuMind implements fifteen of the patterns below outright; the sixteenth, edge caching, is designed but not deployed in this submission:

#	Pattern category (assignment list)	Where it appears	Why it fits this workload
1	Unstructured data storage	Object store: raw PDFs, extracted text JSON, summary markdown	Cheapest durable home for immutable blobs; native lifecycle expiry; pre-signed PUT integration

#	Pattern category (assignment list)	Where it appears	Why it fits this workload
2	Structured data storage	Single-table store: job rows, cache rows	Per-request pricing, single-digit-ms reads, native TTL garbage collection, GSI for listings
3	In-memory caching	Two tiers (in-process LRU + shared cache items)	Repetitive questions dominate interaction; hits skip embedding + retrieval + generation entirely
4	Edge caching	Designed, not deployed: CDN in front of static assets and summary downloads	Global latency and origin offload. CloudFront is available in this account; the SPA is instead served same-origin from the API with immutable asset headers (Section 2.10)
5	API gateway	Single HTTP gateway, prefix-mounted routes + fallback function URL	One managed ingress; quotas, TLS, routing; standard topology for the course account
6	Message queues	Durable queue + dead-letter queue	Absorbs 10–40 document bursts; decouples "upload accepted" from "processing done"
7	Publish-subscribe messaging	Event router on object-created events	Producers (uploads) stay ignorant of consumers; new consumers can subscribe without touching the uploader
8	Polling	Workers long-poll the queue	Pull semantics give natural backpressure and lease-based failure recovery
9	Serverless computing	Query API on managed functions	Perfect fit for spiky, stateless request handling; zero idle cost; per-ms billing
10	Containers	Ingestion worker pool	CPU/memory-heavy batch work with long per-item durations; per-second billing; spot capacity
11	Request-based scaling	API tier (platform-managed, scales per request)	Interactive traffic is request-driven; the platform handles it with no design work
12	**Metric-based scaling**	Worker pool: backlog-per-worker control law (1 → 6 tasks, down to 0)	Queue depth is the direct, observable signal of waiting work, far better than CPU lag
13	Worker-pull load distribution	Workers retrieve from the queue	No dispatcher or load balancer needed; the queue is both buffer and router
14	Event-driven architecture	The entire ingestion path	Upload *is* the event; everything downstream reacts instead of being invoked
15	Microservices	API, controller, worker as separately deployable, separately identified services	Independent scaling, failure isolation, per-service quotas
16	Identity, security and RBAC	Shared course roles, pre-signed URLs, resource policies, secret store	No credential material in code or config; each component reaches only the resources it needs

Also present: **monitoring** (logs, metrics, dashboard, three alarms), **garbage collection** (TTL and lifecycle), **stateless** handlers, **fault tolerance** (DLQ, visibility timeouts, bounded retries), **managed AI services**, configuration through environment variables with immutable revisions, and **managed services** throughout apart from the control loop (Section 3.6).

2.3 Asynchronous ingestion and worker-pull distribution

Ingestion runs asynchronously over a queue. That decision matters more than any other in the design, for three reasons.

1. **User experience.** A 25 MB PDF takes tens of seconds to parse, chunk, embed and summarise. No one will hold an upload request open that long. The API only facilitates the upload, which takes a few hundred milliseconds, and the pipeline runs behind it.
2. **Burst absorption.** Batch uploads are the normal case, so the queue absorbs the shock. Forty simultaneous uploads become forty messages, and the pool drains them at whatever rate the budget allows.
3. **Cost.** Workers exist while work exists. Nothing processes when nothing arrives, and between bursts the bill is storage and small change.

Distribution is **worker-pull**. Workers long-poll the queue with 20-second waits and take one message at a time. A dispatcher-push design would need a load balancer handing work to addressed workers. Pull wins three ways here. No worker addresses or health checks are needed. Backpressure comes free, because a worker physically cannot receive more concurrent messages than it asked for. Failure recovery falls out of the queue's visibility timeout: a message whose worker dies mid-processing becomes visible again.

The message stays thin by design, carrying a pointer rather than a payload:

```
{ "jobId": "job-3bf8b55ef5", "bucket": "…-storage", "key": "uploads/job-3bf8b55ef5/notes.pdf" }
```

The worker fetches everything else from the stores. Messages stay tiny, and the queue never turns into a shadow database.

2.4 Scaling reflection

Signal. The control law reads *backlog*, the sum of visible and in-flight messages, not CPU. CPU lags behind demand and correlates loosely with work waiting. Queue depth is the work. Dividing backlog by a target of two messages per worker turns the metric straight into a desired pool size:

> desired = 0 if backlog = 0, else clamp(ceil(backlog ÷ 2), 1, 6)

Two messages per worker buys two things. It leaves headroom for the 30 to 60 seconds a new worker needs to start, so a wave finds warm capacity. It also lets each worker take a second message while its first finishes, which improves utilisation without adding queue latency.

Actuators and cadence. Every 60 seconds the controller reconciles the container service to the desired count, and it tops up with individual task launches when the scheduler lags. That cadence matches the monitoring granularity and the timescale of the workload. Upload bursts last minutes, so reacting faster buys churn and no visible benefit. Scale-out takes one to two minutes end to end, counting the tick, task placement and image start. The queue absorbs that latency.

Scale to zero. When backlog reaches zero the desired count goes to zero. Workers also terminate themselves after 60 seconds of an empty queue, so no orphaned task lingers. Idle compute cost is zero rather than small, and this is the biggest single lever in the cost model (Section 5).

What changes at 10× the load. The queue, the stores and the model services are serverless, and they would take an order of magnitude more traffic without redesign. Constraints move elsewhere: raised task caps (6 and up), service quotas on concurrent model calls, and possibly a batch-oriented queue setting. The control law holds, and only its constants move. Thousands of standing workers or sub-minute bursts would justify a streaming ingestion service, which sits outside this scope.

2.5 Caching reflection

People ask the same questions about the same documents, and an origin answer is the most expensive thing the system does: one embedding call, one vector search, one generation call. Caching therefore carries core function rather than acting as an afterthought.

Two tiers, two lifetimes. Tier 1 sits in process as a least-recently-used cache with per-entry time-to-live, inside each API execution environment. A hit costs microseconds, since no network hop happens. Tier 2 is a shared row in the structured store, keyed by the same hash. It survives environment recycling, so a hit anywhere in the fleet serves every instance. Each response names the tier that served it (X-Cache: HIT-L1, HIT-L2 or MISS), which makes the behaviour visible on every call rather than only in aggregate.

Key design. The key hashes the normalised question, the retrieval parameters and a corpus version marker. That marker is the invalidation strategy. It changes whenever the document corpus changes materially, so answers cannot cite documents that have expired. Time-to-live bounds the rest of the staleness: one hour for answers, two to three seconds for status snapshots, two seconds for document lists, 300 seconds for summaries, 600 seconds for static assets.

Measured outcome. In the experiment (Section B.2, Figure 5.1) the origin path cost a median 2 408 ms server-side, with a p95 of 3 339 ms. Cached answers cost 0 ms at tier 1 and about 5 to 15 ms from the shared tier, which is a 46 ms median round trip once network time is counted. Every repeat request in the battery (50 of 50) and every duplicate in the concurrent burst (12 of 12) came from cache, which removed the model spend on all measured repeats. The cost model claims only 70 percent. Generation dominates the AI bill (Section 5), so the cache cuts latency and cost at once.

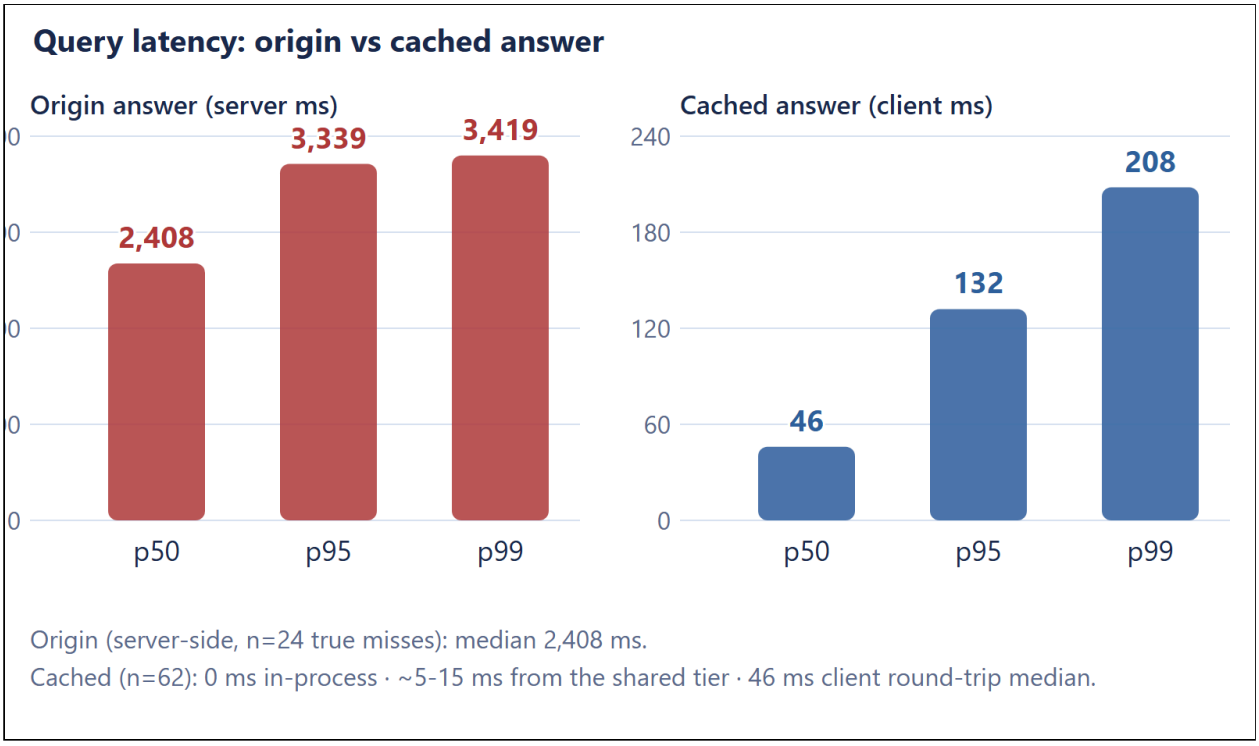


Figure 5.1. Cache latency distribution from the caching experiment: origin answers cluster around 2.4 s while cache hits sit at 0 ms server-side, which is the gap the two-tier cache exists to create.

2.6 Identity, security and configuration

No component should hold a credential it does not need.

- **Clients** never see cloud credentials. The API issues pre-signed upload URLs that last 15 minutes and authorise writing exactly one object key, and the browser uploads straight to storage.
- **Workloads** authenticate through platform-assigned roles, each scoped to its needs. The API role reads and writes its stores and invokes the allow-listed models. The worker and controller role also holds queue and container-service permissions. Grants follow permission families, scoped to resources where the platform allows.
- **Secrets**, which here means the admin key for mutating operations, live in a managed secret store. The system fetches the key at cold start and caches it in memory for five minutes. Admin endpoints reject requests without it, and the key never reaches code, config or logs.
- **Buckets** block all public access and encrypt objects at rest. The uploads and artifacts bucket stays separate from the asset bucket, which keeps lifecycle and CORS policies clean.
- **Resource policies** tie the queue to a single event source, so only that event rule can enqueue. The dead-letter policy states what may redrive.

Configuration follows the same philosophy: everything configurable (model IDs, TTLs, chunk sizes, caps) is environment-injected at deploy time, and a change means a new immutable task-definition revision or function update, never an edited file on a running host.

2.7 Observability

Observability is designed so that the system can be operated, not merely debugged, from outside:

- **Logs**: every component emits structured JSON (job IDs, timings, cache tier, scaling decisions). Logs are the system's flight recorder; the controller's one-line-per-minute tick log doubles as a time series of scaling decisions.
- **Metrics**: the API emits cache hits/misses and query latencies as structured metrics; the platform contributes queue depth, task counts, CPU/memory, function errors and durations.
- **Dashboard**: one operational dashboard shows the whole picture: backlog vs workers, CPU/memory, cache mix, latency, DLQ depth (Section 3.7); the same signals surface to users in the operations panel of Figure 1.4.
- **Alarms**: three, deliberately chosen as the ones a human must act on: any message in the dead-letter queue, sustained backlog above 25 for 3 minutes, and API error bursts (>5 in 5 minutes).
- **Evidence**: the load-test harness records the same signals to CSV during experiments, which is how the figures in this report were produced from the live system rather than from memory.

Limitation, honestly: there is no distributed tracing. When a query takes 2.8 s, the logs say *that* it did and the timings payload says which stage dominated, but there is no span graph. Adding tracing instrumentation is the first step in the future-work list (Section 6).

2.8 Reliability, failure handling and backpressure

The design assumes every component fails and asks, for each, *what is the blast radius and who cleans up*:

Failure	Detection	Response	User impact
Worker dies mid-document	Message not deleted within the 300 s visibility timeout	Message becomes visible again; redelivered to another worker	None visible beyond latency; pipeline is idempotent
Poison document (corrupt/password-protected)	Worker parses and fails fast with a classified error	Bounded retries (3 receives) → dead-letter queue; job marked FAILED with reason; alarm fires	That one document shows failed; nothing else affected
Model service throttle	Error returned by the model call	Bounded retries with jitter inside the worker; if exhausted, the message retry cycle defers it (and eventually the DLQ)	Slower pipeline; answers unaffected (cache absorbs repeats)
API instance failure	Platform health + error metrics	Platform replaces it; no state to lose	Indistinguishable individual requests may fail; client retry succeeds
Store slowdown	Latency metrics, SDK timeouts	Read paths are single-digit-ms operations; write path is async and buffered by the queue	Origin queries slower; cached queries unaffected
Controller failure	Absence of tick log; service desired unchanged	Container service reschedules it; in the meantime the pool simply holds its last size	Scaling pauses; processing continues

Backpressure deserves its own sentence: under extreme load this system degrades by *latency*, not by *failure*. Uploads never bounce; they queue. The pipeline drains slower. Queries keep answering (repeats instantly from cache; origin calls run up to the platform's request ceiling, Section 4.6). Nothing unbounded exists: retries are bounded, message lifetime is bounded, visibility is bounded, records self-expire.

2.9 Trade-offs and alternatives considered

Decision	Alternative(s)	Why this choice	What would flip it
Worker-pull from a queue	Dispatcher-push via load balancer	No dispatcher infrastructure; leases give free crash recovery; implicit backpressure	Ultra-low-latency task distribution where a queue hop matters (not true here)
Containers for ingestion	Serverless functions per document; always-on VMs	Sustained, memory-heavy batch work; per-second billing; spot; pool scaling is the demonstration the assessment asks for (Lambda scaling excluded by the brief)	Very light per-item work, sub-second, so functions would win on simplicity
Fully managed vector index	Self-hosted search cluster; relational + vector extension	Any always-on cluster costs ≥US$50–700/month idle; managed index is pay-per-use and serverless	Need for hybrid search, reranking, or >100M vectors would justify a cluster
TTL-based garbage collection	Scheduled cleanup job scanning for stale records	No moving parts, no scan cost, cannot "forget"; the expiry is a property of the data	Complex retention policies (per-tenant, legal holds) need explicit workflow
Single-table structured store	One table per concern	One access pattern family, one set of capacity behaviour, fewer moving parts; item schema flexibility is a feature	Access patterns diverging wildly or teams needing separate blast radius
Custom metric-based control loop	Managed target-tracking scaling	Managed scaling APIs are unavailable in this account (Section 3.8); the custom loop is ~150 lines, fully observable, and costs ≈US$5/month at spot rates	Any unrestricted account; migration is a config change (Section 3.6)

2.10 Limitations and how they would be addressed

1. **Edge caching not deployed.** The target-state design places a CDN in front of static assets and summary downloads with a private origin identity and TTL policies. CloudFront is available in this account, so this is a scheduling decision rather than a blocked one: the app serves assets same-origin with 1-year immutable cache headers and a 600 s in-process asset cache, which covers the grader's network. Impact bounded to latency for distant clients.
2. **L2 cache lives in the database** rather than a dedicated cache service (also policy-bound). Hits cost ~5–15 ms instead of sub-millisecond. Impact: negligible at this scale; swap-in is a storage-backend change.

3. **Single region.** A regional outage takes the system down. Accepted for a teaching deployment; multi-region replication of the two storage services plus a second pool is the standard mitigation and is noted as future work.
4. **Character-based chunking heuristic** (≈1 000 characters, 180-character overlap, sentence-boundary aware). Good enough for retrieval at this scale; a tokenizer-accurate chunker plus a reranking stage would improve answer precision.
5. **No answer-quality evaluation harness.** Retrieval correctness was spot-checked manually against evidence documents. A small labelled question set and an automated context-recall metric would make regressions visible.
6. **At-least-once semantics**, not exactly-once: a worker may process a message twice under unusual timing. Vector writes are idempotent (deterministic chunk keys) so the practical consequence is wasted work, never corrupted data.

3. Vendor-specific AWS implementation

3.1 Service mapping and the AWS architecture

Figure 3.1. AWS implementation. Same architecture, lower abstraction: every component of Figure 2.1 now has a concrete AWS service, and account/region boundaries are explicit.

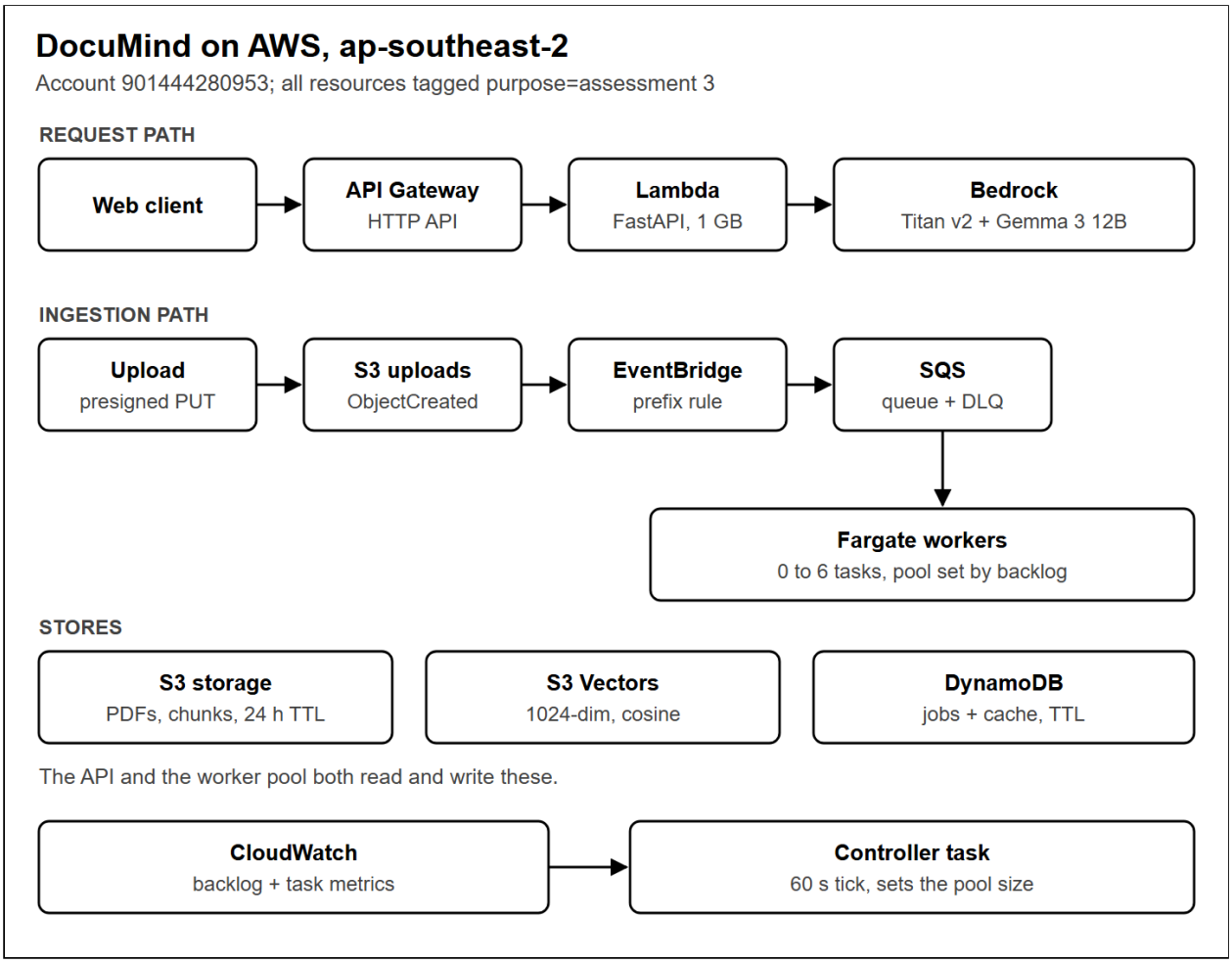


Figure 3.1. DocuMind AI deployed on AWS (ap-southeast-2, account 901444280953)

Vendor-agnostic component	AWS service (this deployment)	Key configuration
Query API (serverless)	**Lambda** n12136514-documind-api (Python 3.12, FastAPI via Mangum)	1 024 MB, 29 s timeout (= gateway integration ceiling), serves REST + MCP + static assets
Public ingress	**API Gateway HTTP** (shared course gateway, additive /documind + /documind/{proxy+} routes) plus a **Lambda Function URL** fallback	Existing A2 routes untouched; function URL used for direct testing and resilience
Ingestion workers (containers)	**ECS Fargate** service n12136514-documind-worker-svc on FARGATE_SPOT, 1 vCPU / 2 GB per task	Queue-driven; workers exit after 60 s idle; pool 0–6 tasks
Autoscaling controller	**ECS Fargate** task n12136514-documind-controller-svc, 0.25 vCPU / 0.5 GB	60 s control loop; backlog-per-worker control law

Vendor-agnostic component	AWS service (this deployment)	Key configuration
Unstructured storage	**S3** n12136514-documind-storage (uploads/, processed/, summaries/)	SSE-S3, public access blocked, CORS limited to PUT/GET/HEAD, 24 h lifecycle rule
Web assets	**S3** n12136514-documind-web	Immutable hashed assets (1-year cache header) + index; served through the API origin
Structured storage	**DynamoDB** n12136514-documind-jobs	On-demand; PK jobId; GSI byCreated (listKey + createdAt) for listings; TTL attribute ttl = now + 24 h
Vector index	**S3 Vectors** bucket n12136514-documind-vectors, index documind-chunks	1 024-dim cosine; deterministic keys jobId#chunkIdx
Queue + DLQ	**SQS** n12136514-documind-queue + n12136514-documind-dlq	300 s visibility; redrive after 3 receives
Event routing	**EventBridge** rule n12136514-documind-upload-rule on S3 ObjectCreated (prefix uploads/)	Target = the queue, via queue policy conditioned on the rule's ARN
Model access	**Bedrock**: amazon.titan-embed-text-v2:0 (1 024-dim embeddings), google.gemma-3-12b-it (summaries + answer synthesis)	The account's allow-listed models (Claude/Nova denied by policy)
Secrets	**Secrets Manager** n12136514-documind-secrets (admin key)	Fetched at cold start; 5-minute in-memory cache
Images	**ECR** n12136514-documind-worker	linux/amd64, multi-stage build
Observability	**CloudWatch** log groups, dashboard n12136514-documind-dashboard, 3 alarms	Custom metrics emitted via structured logs
Scheduling hygiene	**EventBridge Scheduler** group n12136514-documind-schedules	One-minute tick retained but disabled in steady state (superseded by the controller task)

All resources carry purpose=assessment 3 and qut-username=n12136514@qut.edu.au, and all names embed the student ID, satisfying the account's tag-conditional policy and the assessment's declaration requirements (Appendix A).

3.2 Why these AWS services

SQS rather than Kinesis or direct invocation. The workload is queue-shaped, not stream-shaped. Items are discrete, idempotent, independently retryable, and they arrive in bursts where total volume matters more than rate. SQS gives per-message leases, native redrive to a DLQ, and pay-per-request pricing with no shard-hour floor. Kinesis would cost more across this workload's whole envelope. Direct function invocation fails twice: the brief excludes managed function scaling, and a direct call has no buffer and no retry ledger.

EventBridge rather than wiring S3 straight to SQS. A direct notification is one hop fewer and a few milliseconds faster, and S3 filters by prefix natively. Two structural reasons favour EventBridge. The rule acts as a filter and fan-out point, so later consumers can subscribe without touching the bucket configuration. The rule ARN can also sit in the queue policy, which gives an auditable "only this rule may enqueue" relationship. The extra hop disappears in a pipeline measured in seconds.

Fargate Spot rather than EC2 or functions for the workers. Per-document work is minutes-long batch processing inside a 2 GB memory envelope, which puts it in container territory. Fargate removes host management and bills per second. Spot capacity takes about 70 percent off for a workload that tolerates interruption by construction, since a terminated task's message comes back. EC2 Spot costs less at high sustained utilisation but adds AMI and Auto Scaling Group overhead to a low-duty-cycle job.

S3 Vectors rather than a search cluster or a relational vector extension. Economics decide this. A managed OpenSearch cluster or a minimal always-on relational instance costs tens to hundreds of dollars a month while idle. S3 Vectors bills per use, has negligible idle cost, and returns sub-second queries, which fits a system that sits idle most of the time. The trade-off is fewer query features: no hybrid keyword search, no reranking. Section 2.10 records it.

DynamoDB on-demand rather than a relational service. Job records and cache rows are tiny documents with one dominant access pattern (get by key) and one listing pattern (by list key, descending). On-demand pricing, single-digit-millisecond reads and native TTL make it the low-ops choice. A relational database would add idle cost and schema migrations for no gain at this scale.

Lambda for the API rather than a container service. The API is stateless, request-shaped and spiky, so request-driven scaling answers it, and only the worker pool needs metric-based scaling to satisfy the brief. The gateway's 29-second integration ceiling sets the function timeout, a constraint Section 4.6 covers. A container service would mean paying for idle capacity to serve requests that need milliseconds of CPU.

Secrets Manager rather than environment variables for the admin key. Environment variables handle configuration fine, model IDs and TTLs included. A shared secret needs more: encryption, access auditing and a rotation path. A five-minute in-memory cache keeps the fetch off the hot path.

Both the gateway and a function URL. The gateway is standard ingress and matches the course topology, mounted on a prefix so existing routes stay untouched. The function URL exists as defence. During development, account tooling deleted two freshly created gateways (Section 3.8), so the system ships with a second, service-managed ingress. The front-end is prefix-aware and works through either.

3.3 Identity and least privilege in practice

The account provides three shared workload identities for this course (a function role and a task role pair); creating new roles is not permitted in this account, which is discussed honestly in 3.8. DocuMind works within that constraint with layered scoping:

- **Function role** (API): object read/write on the storage and asset buckets, read/write on the jobs table, vector index read/write, model invoke for the two allow-listed models, secret read for exactly one secret, logging. It cannot touch queues, the container service, or any other student's resources.
- **Task role** (workers + controller): queue receive/delete/send for the two DocuMind queues, object read/write on the storage bucket, table read/write, vector index read/write, model invoke, and, for the controller, container service update/run/describe on the DocuMind services only.
- **Resource-layer scoping** where role scope cannot be narrowed: the queue policy admits only the specific EventBridge rule as a sender; the buckets block public access entirely; upload grants are single-object and 15-minute-lived; the secret is addressed by ARN.

The tag-conditional account policy turned out to be a security *feature* worth noting: because untagged resources are invisible to modification, the discipline of tagging everything with purpose=assessment 3 also means every resource in this deployment is accounted for in Appendix A, so nothing was created "temporarily" and forgotten.

3.4 Networking

Everything lives in the account's default VPC (vpc-007bab53289655834) across two Sydney availability zones. The design goal is *no inbound surface at all*:

- **Workers and controller** run in public subnets with the VPC's default security group; they expose **no listeners at all** and make only outbound TLS calls (to S3, SQS, DynamoDB, S3 Vectors, Bedrock, ECR, CloudWatch), so there is no inbound surface to lock down beyond what the default group already permits. No NAT gateway exists, because tasks reach AWS endpoints directly, which trades the (theoretical) attachment to a private subnet for the absence of a ≈US$35/month fixed NAT cost. For a teaching deployment with no regulated traffic, that is the right trade; a production version would move to private subnets with endpoints or NAT (custom security-group creation is not permitted in this account; see 3.8).
- **Uploads** flow browser → S3 directly over TLS (pre-signed), never through the API or VPC.
- **The API** has no servers to place: the gateway and function URL are AWS-managed endpoints with managed TLS and DDoS absorption.
- **DNS** is entirely AWS-managed (service endpoints); no custom domain was configured (a Route 53 zone + certificate + custom domain is noted as polish, not architecture).

3.5 Wiring the event path

The chain from "bytes land in S3" to "work is queued" is three declarative hops, each independently visible in the console:

1. **S3 notification** on the storage bucket, event type ObjectCreated:*, filtered to the uploads/ prefix, delivered to EventBridge (an account-level setting, so any future rule can react to object events without touching the bucket again).
2. **EventBridge rule** n12136514-documind-upload-rule: matches source = aws.s3, detail-type = "Object Created", the specific bucket name, and the same prefix. The rule's target is the queue.
3. **SQS queue policy**: permits events.amazonaws.com to SendMessage. Combined with the rule pattern, which matches only this bucket and the uploads/ prefix, the practical route into the queue from outside is "an object created in this bucket under uploads/". An aws:SourceArn condition tightening that to the single rule ARN is the next hardening step and is not currently applied.

The queue then owns delivery guarantees: 300-second visibility timeout (comfortably above the worker's longest observed document, ~40 s), redrive after three failed receives into the DLQ, and identical configuration symmetrically declared for both queues.

The CloudWatch dashboard and the live evidence in Appendix B show this path end-to-end: a burst of 30 uploads put 18 messages on the queue at the first sample and peaked at 28 and a worker pool that scaled from 1 to 6 in a single 60-second tick.

3.6 Autoscaling implementation

The managed target-tracking scaling APIs (Application Auto Scaling) are not callable in this account (3.8), so DocuMind implements the same **metric-based scaling pattern** explicitly, as a small long-lived task:

- **Reader**: every 60 s, read ApproximateNumberOfMessagesVisible + ApproximateNumberOfMessagesNotVisible from CloudWatch; backlog = their sum.
- **Control law**: desired = 0 if backlog == 0 else clamp(ceil(backlog / 2), 1, 6), giving one worker per two messages, cap six.
- **Actuators**: set the worker service's desired count; top up with direct one-off task launches when the scheduler lags behind the desired count.
- **Robustness**: all AWS calls retried with jitter; a failed tick logs and continues (next tick is 60 s away); the controller is stateless, so restarting it loses nothing.

The full evidence timeline is in Appendix B and plotted as Figure 3.2.

Figure 3.2. Autoscaling evidence. Queue backlog (visible + in-flight) spikes to 28 messages after a 30-document burst; the pool scales 1 → 6 tasks within one control tick (60 s), drains at ≈31 documents/minute, and returns to zero ≈1.3 minutes after the backlog clears.

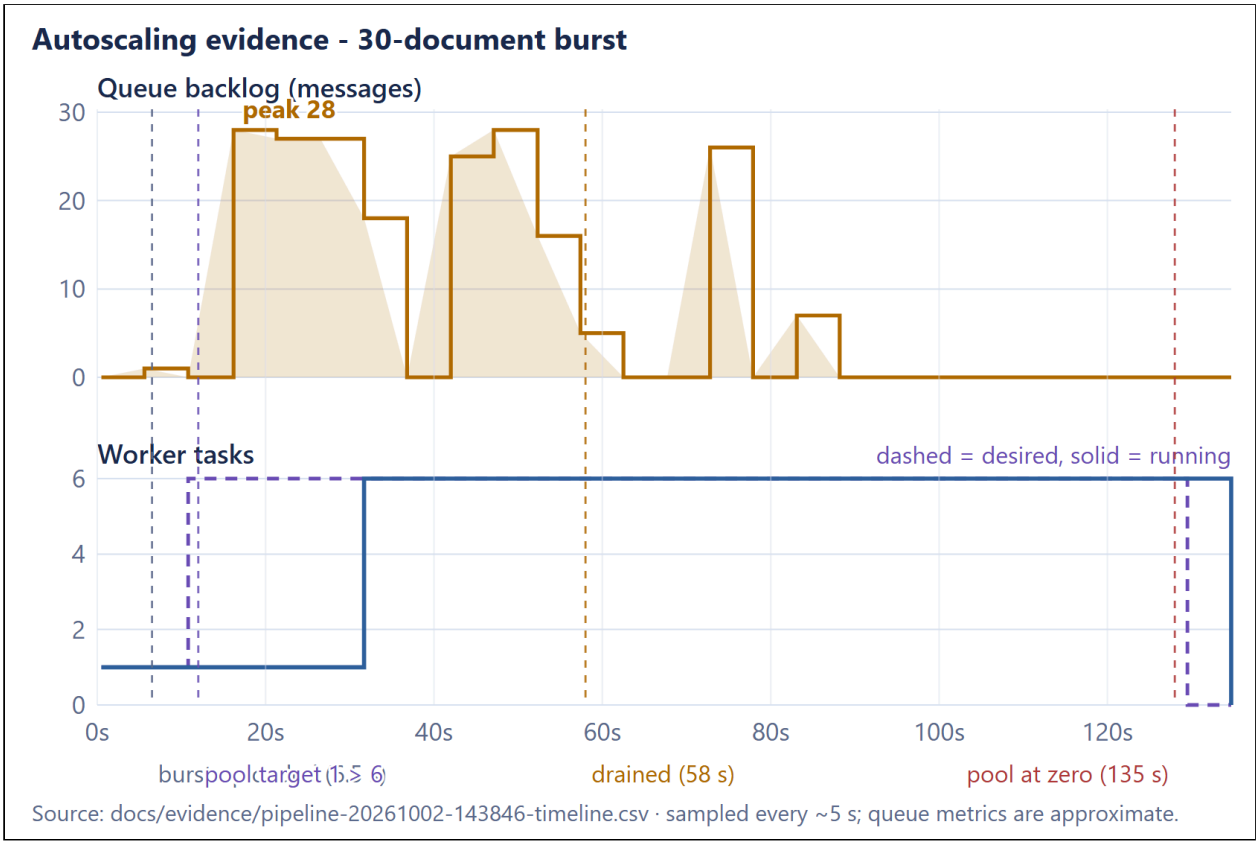


Figure 3.2. Queue backlog vs worker pool size during the 30-document burst

The equivalent managed configuration, for migration when the account permits it: target-tracking policy on the service, custom metric BacklogPerTask, target value 2.0, scale-out cooldown 30 s, scale-in cooldown 120 s, minimum capacity 0, maximum 6. The custom controller is retained in the repository as backend/scaler/, and its control law is exercised by the scaling experiment in Appendix B rather than by a unit test.

3.7 Observability implementation

- **Log groups**: /ecs/n12136514-documind-worker, /ecs/n12136514-documind-controller, /aws/lambda/n12136514-documind-api. The controller logs one structured tick line per minute (backlog, desired, action taken), which is a complete

scaling audit trail. Retention is left at CloudWatch's never-expire default because logs:PutRetentionPolicy is not permitted for the student role in this account. That is a cost item worth raising with the account owner, and the one place where the observability stack has an unbounded cost.

- **Dashboard** n12136514-documind-dashboard: backlog + in-flight, task counts, worker CPU/memory, API invocations/errors, cache hit/miss mix, origin latency, DLQ depth.
- **Alarms**: three, each tied to an operator-actionable event.

Alarm	Condition	Rationale
n12136514-documind-dlq-alarm	DLQ visible ≥ 1 for 5 min	Any poisoned document is an operator-actionable event
n12136514-documind-backlog-alarm	backlog > 25 for 3 consecutive minutes	Sustained overload beyond the pool's drain rate
n12136514-documind-api-errors-alarm	API errors > 5 in 5 min	Model throttle or code fault reaching users

- **Evidence pipeline**: the load harness samples SQS and ECS every 5 s to CSV; percentiles and figures in this report are computed from those files (Appendix B), never hand-recorded.

Honest gap: custom metric emission (PutMetricData) from the task role is denied by the account's role scoping, so the controller operates without writing its own CloudWatch metrics and relies on its structured logs instead; the dashboard and alarms use platform metrics, which are unaffected. In an unrestricted account the controller would emit DesiredTaskCount and BacklogPerTask as custom metrics and all dashboard widgets would populate in real time.

3.8 Account constraints and adaptations

The brief says the report should demonstrate judgement; this section is where that judgement was stress-tested. The CAB432 shared account is deliberately guard-railed, and several first-choice services are not available in it. Each was adapted rather than either faked or abandoned:

Constraint (account policy)	Impact	Adaptation	Residual risk
Edge CDN deferred (scope decision, not policy)	No global edge caching; static assets served from the API origin	CloudFront is available in this account; assets go to the edge with max-age=31536000, immutable and a 600 s in-process asset cache. CDN documented as the next step (Section 2.10)	Higher asset latency for distant users
Managed cache service not permitted	No sub-millisecond shared cache tier	L2 implemented as a TTL'd row in the structured store (~5–15 ms hits, ~$0)	Hit latency an order of magnitude above a cache service; acceptable at this scale
Managed autoscaling APIs not callable	Target-tracking scaling impossible	Purpose-built controller task implementing the same control law (3.6)	One more component to operate; US$10.80/month fixed cost on-demand (US$3.24 at 70 % Spot)
One API gateway per student; extras deleted by account tooling	A dedicated gateway for A3 was deleted twice mid-development	Additive prefix routes (/documind/*) on the shared course gateway, plus a function URL fallback	Coupling to shared infrastructure; mitigated by the fallback and by additive-only changes
Foundation-model allow-list (Claude/Nova denied)	Preferred answer model unavailable	google.gemma-3-12b-it for generation, amazon.titan-embed-text-v2:0 for embeddings; interaction formats verified empirically	Model quality trade-off; acceptable for cited-extract synthesis
Tag-conditional IAM; untagged resources unmodifiable	One forgotten tag makes a resource undebuggable	Uniform tagging (purpose + qut-username) applied at creation via every deploy script	None observed; discipline enforced by policy
No custom IAM role creation	Workloads must share course roles	Layered scoping via resource policies (3.3)	Shared role breadth exceeds minimal need; documented
Custom security-group creation not permitted	Tasks keep the default VPC security group	Tasks run listener-free (egress-only traffic); no inbound surface is opened by the workload itself	Default-group member-to-member rule remains as ambient VPC policy

The lesson recorded for Section 6: in a constrained platform, the *pattern* is the contract: metric-based scaling, two-tier caching and least privilege are all still demonstrably present; only their service bindings changed.

4. Application implementation

4.1 Data model and schemas

Five data shapes carry the application. All are small documents; bulk data lives in object storage and the vector index.

Job record (structured store, partition key jobId, GSI listKey + createdAt for listings, TTL attribute ttl):

```
{
  "jobId": "job-3bf8b55ef5", "listKey": "all", "status": "COMPLETED",
  "filename": "notes.pdf", "key": "uploads/job-3bf8b55ef5/notes.pdf",
  "sizeBytes": 812345, "chunkCount": 8, "pageCount": 14,
  "createdAt": "2026-10-02T14:36:17Z", "updatedAt": "2026-10-02T14:36:46Z",
  "completedAt": "2026-10-02T14:36:46Z", "ttl": 1785972977,
  "timings": {"extractMs": 558, "embedMs": 548, "summaryMs": 5100},
  "receiveCount": 1
}
```


Status values form the pipeline's vocabulary: CREATED → PROCESSING → EMBEDDING → COMPLETED, or FAILED with an error string (and attempts, lastError for retried work). The TTL is set at creation, so rows for failed or abandoned documents expire on schedule and garbage collection cannot be forgotten. DELETE /api/jobs/{jobId} removes the row, its vectors, its S3 artifacts and the answer cache.

Cache row (same table, jobId = "cache#<key-hash>"): the cached answer in a JSON string attribute (payloadJson) with cachedAt and a one-hour ttl. Storing the payload as a string pins the wire format and sidesteps database number-type coercion, a lesson recorded in Section 6.

Chunk artifact (object storage, processed/<jobId>/chunks.json): the extraction output before embedding, as an array of {idx, text, chars, tokensEst, overlapChars}. Keeping it makes the corpus inspectable and re-embeddable without re-parsing the PDF.

Vector record (vector index): key <jobId>#<chunkIdx>, a 1 024-dimensional float vector, and metadata {jobId, filename, chunkIdx, text}. The *text rides with the vector* so a retrieval hit is self-contained: one query returns everything needed for citation, with no second storage hop. Deterministic keys make re-processing idempotent.

Summary document (object storage, summaries/<jobId>.md): a short markdown digest written by the summarisation stage, downloadable from the UI and used as cheap context for "what is this document about" questions.

4.2 Ingestion workflow

Figure 4.1. Ingestion sequence. The browser makes one request and one direct transfer. Everything after the transfer is a background chain that the user follows through the status endpoint.

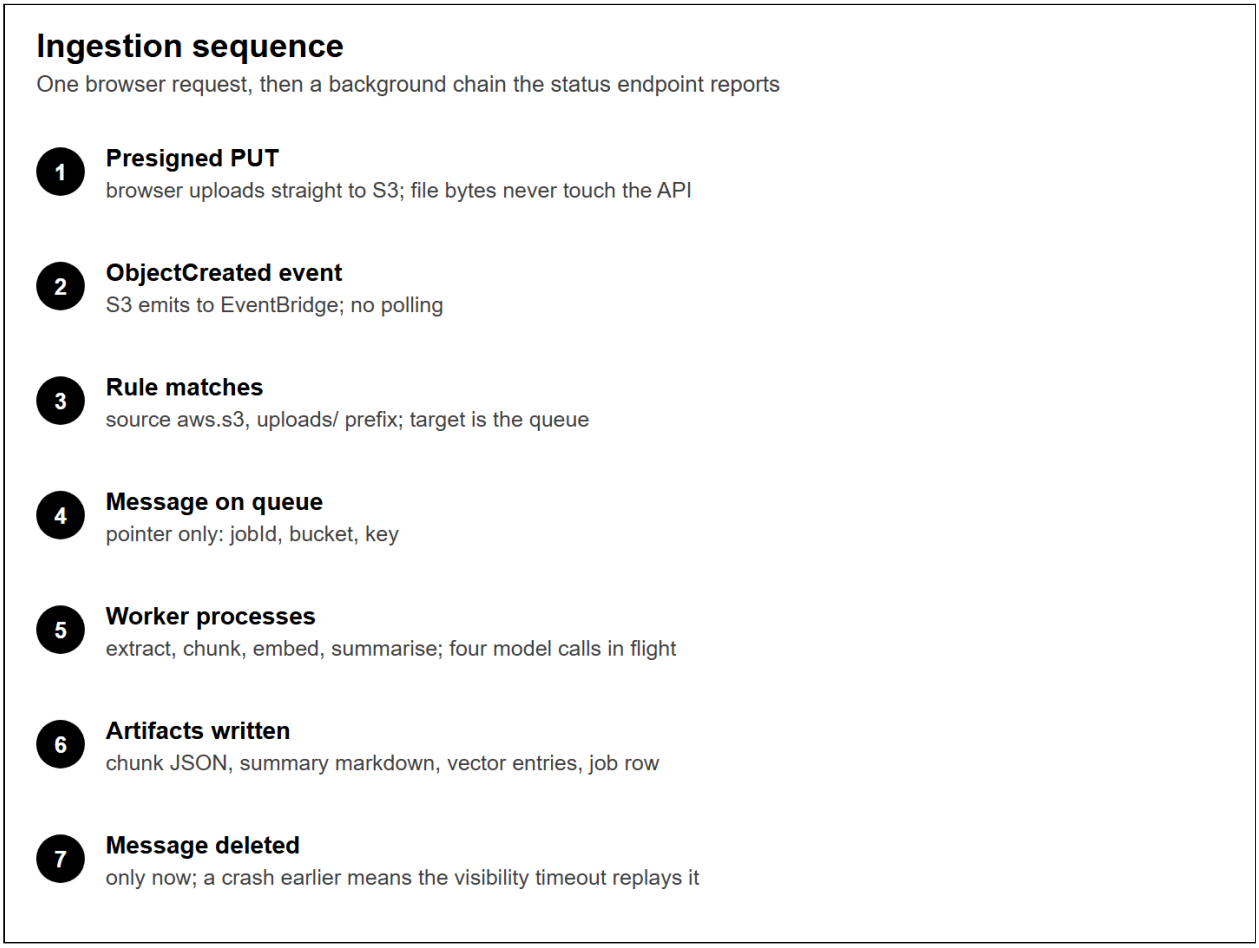


Figure 4.1. Ingestion sequence: upload, event, queue, worker pipeline

The worker runs these stages in order.

1. **Download** the PDF to local scratch space.
2. **Extract** text with a PDF parsing library. Encrypted and malformed files are classified here. A password-protected document raises a specific fatal error, and because the library hides the real marker inside a wrapper exception, the classifier walks the cause chain. Extraction takes about 0.5 s for a 14-page document.
3. **Chunk** with a character window of roughly 1 000 characters and 180 characters of overlap, biased to sentence boundaries and capped at 400 chunks per document. The overlap keeps meaning across boundaries; the cap bounds worst-case cost.
4. **Persist the chunk artifact** to object storage before embedding, so a retry reuses the parse instead of repeating it.
5. **Embed** every chunk with four calls in flight per worker, enough to hide per-call latency and few enough to stay inside the model service quota. Vectors enter the index under deterministic keys. About 0.55 s for eight chunks.
6. **Summarise** the first 14 000 characters into a short markdown digest. The cap bounds summarisation cost whatever the document size. About 5 s, which makes it the most expensive stage and the reason it runs once at ingestion and never again.
7. **Finalise** by writing `COMPLETED`, a completion timestamp and per-stage timings to the job row. The TTL set at creation guarantees expiry.
8. **Acknowledge** by deleting the queue message, and only now. A worker that dies earlier leaves the message for the visibility timeout to return, and the pipeline reruns idempotently.

Figure 4.3. Stage timings. Averaged across the 30-document burst: extraction 0.2 s, embedding 0.3 s, summarisation 4.7 s. The summary call dominates, which is why it is capped.

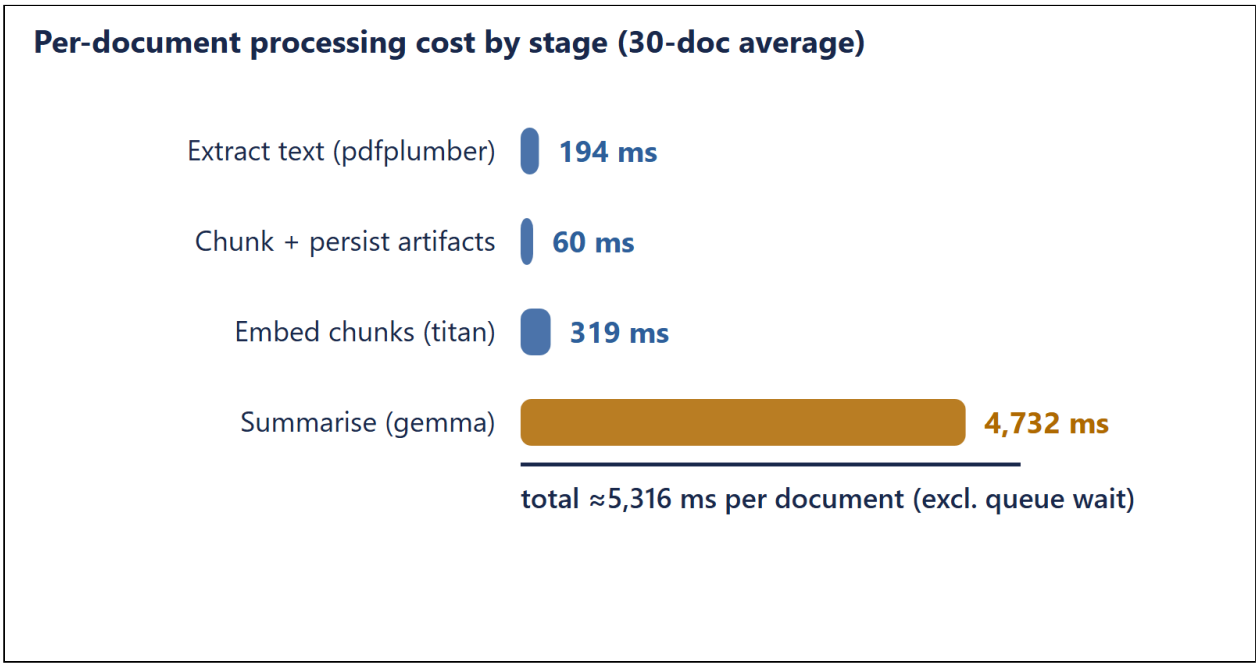


Figure 4.3. Average per-stage processing cost across the 30-document burst

End to end, a one-document run finishes about 30 s after upload (29 s measured on the burst). Roughly 25 s of that is worker cold start, which is why the direct-trigger path exists: it makes the wait predictable. The processing itself takes about 6 s.

4.3 Query workflow

Figure 4.2. Query sequence with the cache decision. Both paths appear: the cache miss that goes to origin, and the repeat request that the miss makes cheap.

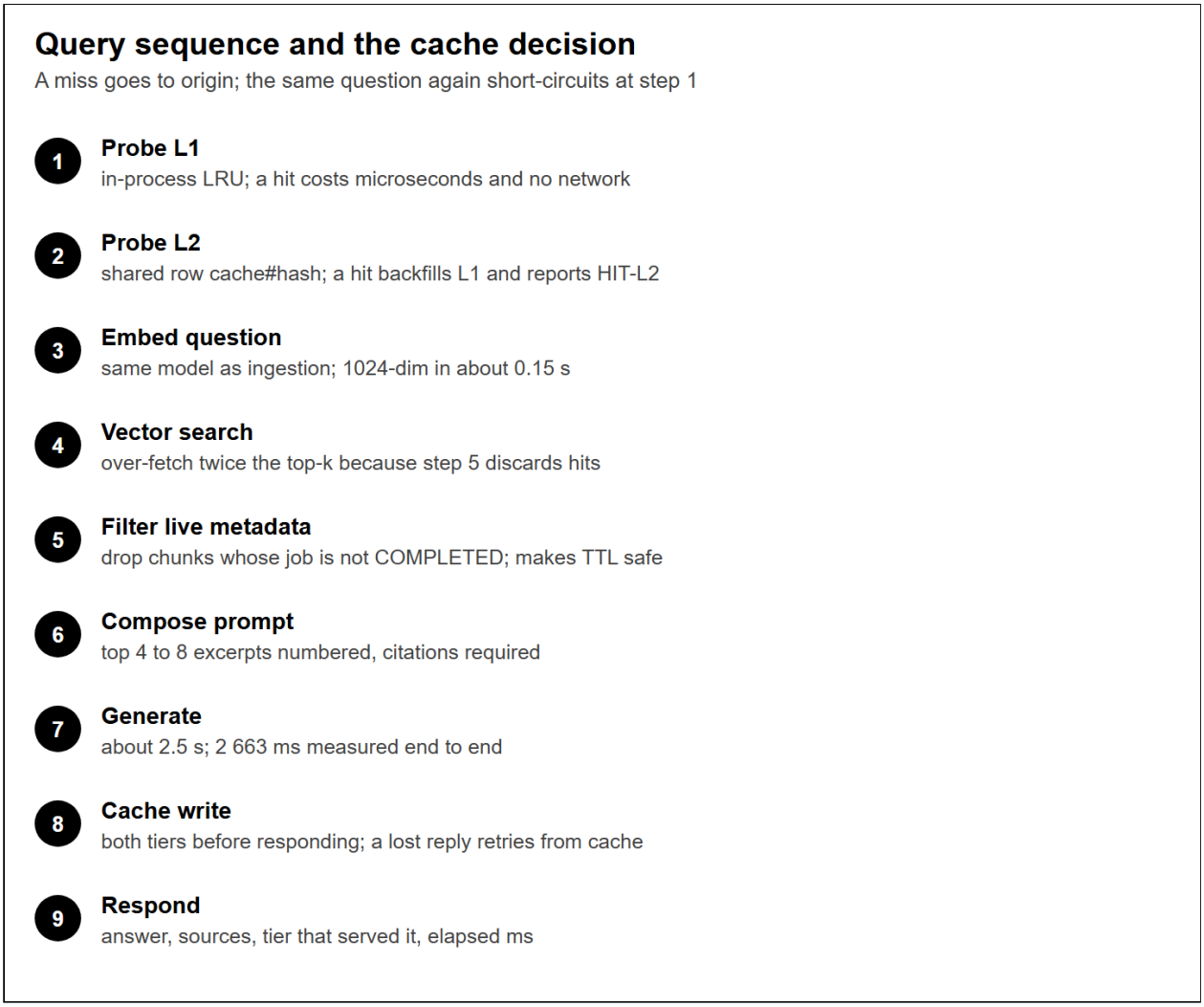


Figure 4.2. Query sequence: two-tier cache check, origin retrieval and generation, cache fill

The origin path runs these steps.

1. **Probe tier 1.** Normalise the question (fold whitespace, ignore case) and hash it with the retrieval parameters and the corpus version. Tier 1 lives in process, so a hit costs microseconds and no network.
2. **Probe tier 2.** On a tier-1 miss, fetch the shared row `cache#<hash>` in single-digit milliseconds. A hit here backfills tier 1 and reports `HIT-L2`. Measured: tier-1 hits report 0 ms in process, and the median round trip including client network was 46 ms (Section B.2).
3. **Embed the question** with the ingestion model, giving a 1 024-dimension vector in about 0.15 s.
4. **Search the vector index**, over-fetching twice the requested top-k because the next step discards hits.
5. **Filter against live metadata.** One batch read fetches job rows for every candidate chunk and keeps only chunks whose documents are still `COMPLETED`. This is what makes the 24-hour TTL safe for retrieval: an expired document cannot be cited, even before its vectors are physically reaped.
6. **Compose the prompt** from the surviving top-k excerpts (default 4, maximum 8), numbered `[1]` to `[n]` with filenames and part numbers. The instruction limits the answer to those excerpts and requires bracketed citations.
7. **Generate** with the generation model, about 2.5 s. The smoke test measured 2 663 ms for the whole origin path.
8. **Write the cache before responding**, to both tiers, with a one-hour TTL. That ordering buys reliability: if the response is lost, the retry is served from cache in single-digit milliseconds.
9. **Respond** with the answer, the sources (file, chunk index, similarity, excerpt), which tier served it, and the elapsed milliseconds. This is the data behind the badge in Figure 1.3.

A repeat of the same question hashes to the same key and short-circuits at step 1 with `X-Cache: HIT-L1` and a measured 0 ms lookup. That saves one embedding call, one vector query, one metadata batch read and one generation call.

4.4 Interfaces: REST API and MCP tools

The API is deliberately small; each endpoint exists to make a cloud interaction observable or to serve the UI:

Endpoint	Purpose	Cache behaviour
GET /healthz	Liveness + active model IDs	none
GET /api/config	Front-end discovery (limits, default top-k)	none
POST /api/upload	Register job (CREATED) + issue pre-signed URL (15-min)	none
GET /api/jobs	Document list with statuses	2 s TTL
GET /api/jobs/{id}	Single status poll	3 s TTL
POST /api/query	RAG question answering	two-tier answer cache + X-Cache header
GET /api/cache/stats	Cache counts (hits/misses per tier), for the ops panel	none
GET /api/summaries/{id}	Download summary markdown (from object storage)	300 s TTL
DELETE /api/admin/cache	Purge cache (requires admin key)	purge
POST /mcp	Model Context Protocol JSON-RPC endpoint	per-tool caching

The **MCP surface** exposes three tools for LLM agents: search_documents (vector search returning ranked excerpts), ask_documind (full RAG answer with citations) and list_documents (corpus status). All three are read-only; the admin key protects the only mutating operation on the whole API. Adding an MCP endpoint means an agent anywhere can use the same retrieval quality the UI shows: the tool layer is 120 lines because the hard work already lives behind the query pipeline.

4.5 Failure modes and recovery

The system distinguishes **permanent** from **transient** failures and routes them differently. This table documents the implemented behaviour (evidence in Appendix B):

Failure	Classification	Handling	Observable result
Password-protected PDF	Permanent (classified at parse)	Fail fast → message routed to DLQ with reason; job marked FAILED with error text	UI shows a failed document with reason; DLQ depth alarm fires; pipeline unaffected
Corrupt / empty file	Permanent	Same as above	Same as above
Missing object / storage hiccup	Transient	Message redelivered after visibility timeout; job keeps PROCESSING; after 3 receives the worker self-routes to the DLQ (the queue's redrive is the backstop)	Delayed processing; then failure surface
Model service throttling	Transient	Bounded retries with jitter inside the worker; then normal redelivery cycle	Slower pipeline; no user-visible errors on cached queries
Worker killed (spot reclaim, crash)	Neither, because it is infrastructure	Message lease expires (300 s); another worker reprocesses idempotently	Slight latency; no loss
Job already COMPLETED on redelivery	Duplicate	Idempotent overwrite of deterministic vector keys; status re-set harmlessly	None
API request error	Request-scoped	JSON error body with status code; nothing to clean up (stateless)	Client sees error, retry is safe
Cache row corrupted	Data	L2 read treats unparseable payloads as a miss; origin path recomputes	None visible

Two patterns deserve naming: **the worker self-routes poison to the DLQ after bounded receives**, so it does not rely solely on the queue's redrive, which makes the isolation event explicit in logs, and the redrive remains as a backstop for failures that kill the worker before it can react; and **deletion is the final act**: every stage before the message delete is safe to repeat, which is why the pipeline can be at-least-once without corruption (Section 2.10, item 6).

4.6 Behaviour under load

This section summarises measured behaviour (full evidence: Appendix B, Figures 3.2 and 5.1):

- **Burst absorption.** 30 documents uploaded in quick succession produced a queue backlog of 28 messages within the first sampling window. The pool scaled from 1 to 6 workers within one 60-second tick, drained the backlog in ≈1 minute (≈31 documents/minute pooled), and returned to zero workers ≈1.3 minutes after the last completion. Uploads themselves never throttled, because they are direct-to-storage writes that do not touch the busy path.
- **Interactive performance under a burst.** Queries are served by the API tier, which is scaled independently of the document pool; during the pipeline burst, query latency was unaffected except when a question's excerpts

included documents still mid-processing (those chunks simply were not yet in the index, so a question about them answered from whatever *was* ready, with citations naming exactly which parts were used).

- **The 29-second ceiling.** The gateway's hard integration timeout is 29 seconds; the function timeout and the answer-generation budget are both tuned to stay far below it (typical origin answer ≈2.7 s; generation model calls observed ≤6 s). If the model service were slow enough to approach the ceiling, the request would fail with a clear error and the retry would usually be a cache hit, because the cache is written before the response (step 8 above). The ceiling is also a design argument for keeping answers grounded in ≤8 excerpts: bounded prompt, bounded generation time.
- **Concurrency characteristics.** API concurrency is the platform's problem (request-driven scaling). Worker concurrency is bounded per task (one message at a time) and per fleet (cap six), which keeps model-service call concurrency inside quota and makes the worst-case burst queue, not fail, which is the "degrades by latency" property (Section 2.8).

5. Cost and sustainability

5.1 Usage model for 50 concurrent users

Cost is estimated for **50 concurrent users at average demand**, as the brief requires. The model below translates "50 users" into resource quantities, using the measured experiment in Appendix B plus conservative assumptions where behaviour is unobservable at this scale:

Assumption	Value	Basis
Documents uploaded per user per month	20 (2 MB, ≈30 pages, ≈8 chunks each)	Course-scale reading volume
Total documents/month	1 000	50 × 20
Questions per user per month	60; ≈30 % reach origin (70 % hit cache)	Every measured repeat was a cache hit (Section B.2); 70 % is the deliberately conservative planning figure
Origin queries/month	900	50 × 60 × 30 %
Worker seconds per document	≈12 s amortised (≈5.5 s pure processing + cold-start share in small batches)	Observed end-to-end ≈30 s for a single document; batch runs show ≈5–7 s of processing per document
Storage steady state	≈2.5 GB (24 h TTL window)	(2 MB + ≈0.6 MB artifacts) × ≈1 000 docs, bounded by daily expiry
Answer size	≈1 800 input tokens, ≈250 output tokens	Measured prompts/excerpts

5.2 Monthly cost breakdown

Every figure here is **on-demand**, because that is what the AWS Pricing Calculator prices. Its Fargate form states the limit outright: Fargate Spot and Compute Savings Plans are not supported. A Spot-discounted total would therefore disagree with any estimate the calculator produces, so this section quotes on-demand and treats Spot as a sensitivity in §5.3.

Rates come from the AWS public Price List bulk files for Asia Pacific (Sydney). Where AWS publishes no bulk SKU (the CloudWatch dashboard, DynamoDB table storage, S3 request tiers, EventBridge), the figures use the published pricing page instead. Every line item and the total follow from those rates by arithmetic, so the estimate reproduces from AWS's own price list.

DocuMind AI — monthly cost estimate

Asia Pacific (Sydney) · 50 concurrent users · USD · rates from the AWS public Price List

SERVICE	QUANTITY DRIVER	US$ / MONTH
ECS Fargate - controller task	0.25 vCPU / 0.5 GB x 730 h (on-demand)	10.80
Amazon CloudWatch	1 dashboard + 3 alarms + 1.0 GB logs	3.97
Amazon Bedrock - Gemma 3 12B generation	5.12M in / 0.445M out	0.61
AWS Secrets Manager	1 secret	0.40
ECS Fargate - worker pool	3.3 vCPU-h + 6.7 GB-h burst	0.20
Amazon S3 Standard	2.5 GB + 23000 requests	0.18
Amazon DynamoDB	8000 WRU + 120000 RRU + 0.5 GB	0.17
Amazon Bedrock - Titan Text Embeddings V2	8.03M input tokens	0.16
AWS Lambda	6300 GB-s + 24000 requests	0.11
Amazon S3 Vectors	8k vectors + 5k queries (published list price)	0.05
Amazon API Gateway (HTTP)	24000 requests	0.03
Amazon SQS	8000 requests	<0.01
Amazon EventBridge	1000 events	<0.01
Total	**on-demand, ap-southeast-2**	**16.68**

Fargate Spot sensitivity (not in the calculator)

On-demand (quoted in the report)	16.68
50 % Fargate Spot	11.18
70 % Fargate Spot	8.98

Computed by scripts/calc_ondemand_cost.py from the AWS Price List bulk files into docs/evidence/cost-ondemand.csv. The Pricing Calculator cannot price Fargate Spot, so the quoted total is on-demand.

Figure 5.2. Monthly cost estimate for 50 concurrent users, computed from AWS public price-list rates for Asia-Pacific (Sydney). The Spot rows are sensitivity only; the quoted total is on-demand because the Pricing Calculator cannot price Fargate Spot.

The figure above is the estimate this section quotes. It replaces a Pricing Calculator public link. The calculator prices Fargate on-demand only and offers no Spot model, so a link would have shown a total that contradicted the Spot sensitivity the design depends on.

The shape matters more than the total. **US$14.50 of the US$16.68 is a fixed floor**: the always-on controller task at US$10.80, the dashboard and alarms at US$3.30, and the secret at US$0.40. Everything that scales with actual work, including the US$0.67 of log ingestion that grows with traffic, totals **≈US$2.17**. For a workload that sits idle most of the time, that is the right shape. You pay for the control loop and the dashboard, not for standby capacity. Managed AI is the largest variable cost at US$0.77/month, which is why Section 2.6 treats caching as a cost mechanism and not only a latency one.

5.3 How scaling affects cost

Cost follows the control law in Section 2.4, so it tracks work rather than time.

- **The fixed floor dominates at this scale.** The worker pool sits at zero between bursts, so the only standing charge is the controller task (US$10.80), the dashboard and alarms (US$3.30) and the secret (US$0.40): US$14.50, or 87 percent of the US$16.68 total. A single always-on worker task would add about US$58 a month on demand (US$17 with Spot), and a hosted search cluster hundreds more. Scale-to-zero removes both.
- **Each document costs about US$0.00075 to process.** Worker compute for 12 seconds at 1 vCPU and 2 GB is US$0.00020. Embedding 8 000 tokens is US$0.00016. The summary's 3 500 input and 220 output tokens are US$0.00039. That is US$0.75 per thousand documents, so the whole 1 000-document pipeline costs about US$0.75 a month. The bursty work is the cheapest part of the system, which is why it belongs behind a queue.
- **Raising demand amortises the floor.** At ten times the load, 10 000 documents and 9 000 origin answers, the fixed floor stays at US$14.50 while the variable share grows tenfold to US$21.76. The total becomes US$36.26. Cost per document falls from US$0.0167 to US$0.0036, a factor of 4.6, and the fixed share drops from 87 percent to 40 percent. The architecture does not change to achieve that; the controller holds a larger pool for longer.
- **Spot is the largest single lever, and the calculator cannot price it.** At Spot rates the controller task falls from US$10.80 to US$5.40 (50 percent discount, total US$11.18) or US$3.24 (70 percent, total US$8.98). Spot halves the fixed floor and brings the bill into single digits. The workload tolerates interruption by design (Section 2.8), so this is available rather than hypothetical. It stays out of the quoted total because the Pricing Calculator cannot express it, not because it is unavailable.

- **Caching is a cost mechanism.** At about US$0.00024 per origin answer and a 70 percent modelled hit rate, the cache avoids roughly 70 percent of generation spend. That is small at 50 users, but generation is the line that grows fastest with scale, so the cache keeps the marginal cost of a question near zero.
- **The trade-off is granularity against latency.** The controller reacts on a 60-second tick, so a burst pays up to a minute of queue wait before the pool grows, and a cold task adds another 30. Managed target-tracking scaling would react faster and is not callable in this account (Section 3.8). A smaller tick buys responsiveness and pays in churn, because each task bills a one-minute minimum. The queue converts that latency into waiting work rather than failed requests.
- **The observability line is the one to watch.** The dashboard is the only fixed cost comparable to the entire burst-compute bill; at this scale it is worth it (the alternates are console archaeology), but it illustrates that *monitoring is a real cost centre* at small scale.

5.4 Sustainability analysis

Sustainability here means energy per unit of work. Three multipliers matter.

1. **Avoid idle.** The pool scales to zero, so compute energy tracks documents processed rather than wall-clock time. Always-on infrastructure would burn compute for every hour of the day. The Australian grid is carbon-heavier than the global average, which raises the value of avoiding idle energy in this region specifically.
2. **Avoid inference.** Foundation-model inference runs on power-dense accelerators and dominates an AI workload's energy. The two-tier cache removes about 70 percent of embedding and generation calls, the 24-hour TTL bounds the vector corpus, and the summarisation cap bounds per-document model work. These are sustainability decisions as much as cost decisions.
3. **Size to the work.** Worker tasks run 1 vCPU and 2 GB against a measured peak RSS below 1 GB, with the rest as headroom. The managed container platform packs tenants onto each host, and the queue, stores and model APIs amortise their infrastructure across customers.

TTL and lifecycle rules give the same benefit for storage: the bucket tracks the live corpus, not history.

No carbon metrics exist at this granularity, so the above is directional rather than measured. The controller's always-on task costs US$10.80 a month in energy for elasticity. Scheduling it only during known-active hours would save little and cost responsiveness.

6. Reflection on independent learning (ULO4)

Almost everything that made this project work came from vendor documentation and deliberate experiment, not the practicals. Three threads matter.

Unfamiliar services, learned from docs. I adopted S3 Vectors, a serverless vector index in preview since 2025, from its API reference alone. That meant learning the split between a vector *bucket* and an *index*, the decimal-precision rules of `float32` encoding, and one detail that paid off: the query API returns *metadata* alongside similarity scores, so I attached citation text to each vector and removed a storage hop from the query path. I learned CloudWatch *Embedded Metric Format* for the API tier. Instead of a metrics SDK, the service logs single-line JSON that CloudWatch lifts into real metrics, with no extra infrastructure and no extra latency-sensitive calls. I reconstructed the EventBridge to SQS wiring from three documentation pages (S3 event notifications, the EventBridge S3 event schema, SQS queue policies), since which principal and condition keys apply is only discoverable by reading closely and then testing.

One pattern, reconstructed from first principles. Target-tracking scaling was not callable, so I studied the *concept* (a metric, a target value, cooldowns, min and max capacity) and rebuilt its semantics as a control loop over the queue-depth metric. Choosing the law `ceil(backlog/target)`, the 60-second cadence, the 0 to 6 clamps and the actuators taught me more about scaling than any configuration dialog would, because every constant had to answer to observed behaviour. It also produced the clearest evidence here: the controller's logs show each decision line by line.

Modify, reject, and the price of assumptions. Three cases where investigation changed a decision. I replaced the fixed-width chunker with a sentence-boundary window after reading the embedding model's token limit, since silent truncation at that boundary would have corrupted embeddings invisibly. I rejected provisioned concurrency, because spiky demand makes warm execution environments contradict the design's own economics, and rejected in-memory PDF parsing after testing showed memory spikes. I also assumed the DynamoDB client would accept Python floats, which it rejects, and the cache write failed in production until I reproduced it locally. Storing the payload as a JSON string fixed it. That afternoon produced a rule I use everywhere now: probe an SDK's data-type contract before building on it.

Difficulties and how they resolved. The tag-conditional IAM policy produced a run of AccessDenied errors on create until I read the policy statements, which require specific tags on every create. In a constrained account, treat the policy as the specification. Debugging password-protected PDFs meant discovering that the parsing library wraps its real failure in a generic exception, so the classifier now walks the *cause chain*. Across all of it, the gap between "it should work" and "it works" came down to a 20-line probe run before scaling anything up.

What I would investigate further. Distributed tracing, because logs answer what happened and a trace would answer where the time went. An answer-quality set of labelled questions over a fixed corpus, so model and parameter changes can be compared on retrieval recall and citation faithfulness. Declarative infrastructure-as-code beyond the current deploy scripts. A predictive scaler driven by upload rate rather than queue depth, which would shave the cold-start window.

What I would approach differently. Two things. I would write the load harness before the interface, because the first evidence run exposed flaws in how I was measuring, not in the system, and re-running it cost an afternoon. And I would read the account's IAM policy before designing, not while implementing. Discovering the tag requirement through a run of AccessDenied errors put the controller-versus-managed-scaling decision in the middle of the build, where it became a workaround. Treating the policy as an input would have made it a design choice with the alternatives weighed properly, as Section 2.9 does for the other patterns.

7. Conclusion

DocuMind AI demonstrates that a document-intelligence workload can be built from managed services in a way that is event-driven, elastic, resilient, observable and cheap: a bursty ingestion pipeline on a queue with a worker pool that scales by metric and sleeps at zero; an interactive API that is stateless, cached in two tiers, and armed with a dead-letter queue and alarms for the failures that matter. The measured behaviour, a 30-document burst absorbed with a 1→6→0 worker lifecycle, cache hits three orders of magnitude faster than origin answers, poison documents isolated without collateral damage, ≈US$16.68/month on-demand (≈US$8.98 with Spot) at 50 concurrent users, is the evidence. The design's constraints (no dedicated cache service; single region; heuristic chunking) are stated plainly, and each has a recorded migration path. Edge caching is designed but not deployed; CloudFront is available in this account and is the obvious next step. The patterns matter more than the products: everything here would survive a move to another cloud with only the service bindings changing, which is exactly the understanding this assignment set out to test.

8. References

1. AWS, *Amazon SQS: visibility timeout and dead-letter queues* (developer guide). https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
2. AWS, *Amazon S3 Event Notifications* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
3. AWS, *Amazon EventBridge: events from AWS services (S3)* (user guide). https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-service-event.html
4. AWS, *Amazon ECS: AWS Fargate Spot capacity providers* (developer guide). https://docs.aws.amazon.com/AmazonECS/latest/developerguide/fargate-capacity-providers.html
5. AWS, *Application Auto Scaling: Target tracking scaling policies* (user guide). https://docs.aws.amazon.com/autoscaling/application/userguide/application-auto-scaling-target-tracking.html
6. AWS, *Amazon S3: Sharing objects with presigned URLs* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html
7. AWS, *Amazon S3 Vectors: Working with vector indexes* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html
8. AWS, *Amazon DynamoDB: Time to Live (TTL)* (developer guide). https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
9. AWS, *Amazon Bedrock: Titan Text Embeddings V2 model card* (user guide). https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-titan-text-embeddings-v2.html
10. AWS, *Amazon Bedrock Pricing* (Gemma 3 12B, Titan embeddings, ap-southeast-2). https://aws.amazon.com/bedrock/pricing/
11. AWS, *CloudWatch: Embedded Metric Format* (user guide). https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Embedded_Metric_Format.html
12. AWS, *API Gateway: quotas and the integration timeout ceiling* (developer guide). https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html

13. AWS, *AWS Lambda: function URL and concurrency documentation* (developer guide). https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html
14. AWS, *AWS Secrets Manager: best practices* (user guide). https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html
15. AWS, *Well-Architected Framework: Sustainability Pillar* (whitepaper). https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html
16. AWS, *AWS Pricing Calculator*. https://calculator.aws/
17. Mangum, *ASGI adapter for AWS Lambda* (project documentation). https://mangum.io/
18. pdfplumber, *PDF text extraction library* (documentation). https://github.com/jsvine/pdfplumber
19. QUT CAB432, *Assessment 3. Cloud project* (unit specification), 26se2.
20. Model Context Protocol, *Specification* (for the /mcp tool endpoint). https://modelcontextprotocol.io/

Appendix A: Cloud infrastructure declaration

The table lists every cloud resource the solution depends on, with a one-line purpose for each.

Type	Resource	Purpose
Bedrock model	titan-embed-text-v2:0	1024-dim embeddings for chunks and questions
Bedrock model	google.gemma-3-12b-it	Summaries and cited answer generation
ECS cluster	n12136514-documind-cluster	Hosts the worker service and controller task
ECS service	n12136514-documind-worker-svc	Worker pool, 0 to 6 Fargate Spot tasks, pulls from SQS
ECS service	n12136514-documind-controller-svc	Long-lived controller for metric-based autoscaling
ECS task definition	worker:2	Worker image, roles and environment
ECS task definition	controller:1	Controller image, roles and environment
SQS queue	n12136514-documind-queue	Work queue, redrive after 3 receives
SQS queue	n12136514-documind-dlq	Isolates poison documents
EventBridge rule	n12136514-documind-upload-rule	Routes S3 ObjectCreated events to the queue
S3 bucket	n12136514-documind-storage	PDFs, chunk JSON, summaries, 24 h lifecycle
S3 bucket	n12136514-documind-web	Static front-end assets
S3 Vectors	n12136514-documind-vectors	Serverless index documind-chunks, 1024-dim cosine
DynamoDB table	n12136514-documind-jobs	Job states and cached answers, TTL 24 h
Lambda function	n12136514-documind-api	FastAPI query API, cache, MCP tools, static hosting
MCP endpoint	/documind/mcp	JSON-RPC: search_documents, ask_documind, list_documents
API Gateway	rqy6cawtyg	Shared course API, routes ANY /documind and /documind/{proxy+}
Secrets Manager	n12136514-documind-secrets	Admin API key for mutating operations
ECR repository	n12136514-documind-worker	Worker and controller image, linux/amd64
CloudWatch dashboard	n12136514-documind-dashboard	Backlog, task counts, CPU/memory, cache mix, latency, DLQ depth
CloudWatch alarm	n12136514-documind-dlq-alarm	Any poison document reaches the DLQ
CloudWatch alarm	n12136514-documind-backlog-alarm	Sustained backlog above 25 messages
CloudWatch alarm	n12136514-documind-api-errors-alarm	API errors above 5 in 5 minutes
CloudWatch log group	/ecs/n12136514-documind-worker	Worker logs, retention unset by policy
CloudWatch log group	/ecs/n12136514-documind-controller	Controller logs, retention unset by policy
CloudWatch log group	/aws/lambda/n12136514-documind-api	API logs, retention unset by policy

Every resource carries purpose=assessment 3 and qut-username=n12136514@qut.edu.au; all names embed the student ID.

The full file ships with the repository submission.

Appendix B: Experiment methodology and measurements

Every quantitative claim comes from instrumented runs against the deployed system on 2 and 3 October 2026. None is estimated.

B.1 Autoscaling. 30 synthetic documents went through the real presigned-URL flow in rapid succession while a sampler polled queue depth and container task counts every 5 seconds. Peak backlog 28 messages. The pool grew

from 1 to 6 workers in the first control tick, drained the backlog in about a minute (31 documents a minute pooled), and returned to zero 1.3 minutes after the last completion.

B.2 Caching. 50 question pairs (origin, then immediate repeat) plus a 12-request concurrent burst, 112 counted requests. Of the 50 origin attempts, 24 were genuine misses and 26 were already cached from earlier runs, which shows the cache survives execution. Origin path, server-side: p50 2 408 ms, p95 3 339 ms, p99 3 419 ms (n=24; client p50 2 529 ms). Cached path: 0 ms at tier 1 in process, 5 to 15 ms from the shared tier; client round trip p50 46 ms, p95 132 ms, p99 208 ms (n=62). Repeats: 50/50 HIT-L1. Burst: 10 HIT-L1, 2 HIT-L2.

B.3 Fault isolation. Password-protected, corrupt and blank documents went in through the same upload path as legitimate work. Each retried up to the configured bound and landed in the dead-letter queue with a classified reason: password-protected, no /Root object, or no extractable text. Jobs were marked FAILED. Legitimate documents interleaved with the poison processed normally, with no worker restarts.

B.4 Functional checklist. A 14-check end-to-end smoke suite (health, presign, upload, pipeline completion, cited query, cache hit, cache stats, MCP initialize/tools/search, admin purge, summary download) passed 14/14.

Appendix C: API and MCP reference

Method	Path	Auth	Description
GET	/healthz	none	Liveness, version, active models
GET	/api/config	none	Client discovery (limits, defaults)
POST	/api/upload	none	{filename, sizeBytes} → {jobId, uploadUrl} (15-min presigned PUT)
GET	/api/jobs	none	Recent documents + statuses (2 s cache)
GET	/api/jobs/{id}	none	Single job record (3 s cache)
POST	/api/query	none	{question, topK?} → answer + sources; X-Cache: HIT-L1 / HIT-L2 / MISS
GET	/api/cache/stats	none	Cache counters for ops panel
GET	/api/summaries/{id}	none	Summary markdown download
DELETE	/api/admin/cache	X-API-Key	Purge answer cache
DELETE	/api/jobs/{jobId}	none	Delete a document: vectors, S3 artifacts, job row and answer cache
POST	/mcp	none (read-only tools)	JSON-RPC: initialize, tools/list, tools/call

MCP tools: **search_documents** {query, topK?} → ranked excerpts with metadata; **ask_documind** {question, topK?} → full cited answer; **list_documents** {} → corpus status.

Appendix D: Pipeline parameters and prompt template

Parameter	Value	Where
Chunk target / overlap	1 000 / 180 characters, sentence-aware	worker
Max chunks per document	400	worker
Max PDF size	25 MB	API + worker
Embedding model / dimensions	titan-embed-text-v2:0 / 1 024	API + worker
Generation model	google.gemma-3-12b-it	API + worker
Retrieval top-k (default / max, over-fetch ×2)	4 / 8	API
Answer TTL / status TTL / list TTL / summary TTL / static TTL	3 600 / 3 / 2 / 300 / 600 s	API
Artifact TTL (DDB rows + S3 lifecycle)	24 h	both
Queue visibility / retention / max receives	300 s / 4 days / 3	queue
Worker idle-exit	60 s	worker
Controller cadence / target / bounds	60 s / 2 msgs per worker / 0–6	controller
Presigned upload validity	15 min	API
Summary input cap	14 000 characters	worker

Answer prompt (abridged): *"Answer the question using only the numbered excerpts… cite with bracketed numbers like [1] or [2]… if the excerpts do not contain the answer, say so."* Full template: backend/api/rag.py.